Group Assessment Coversheet

To be attached to the front of the assessment.

Campus: EDUVOS MIDRAND

Faculty: INFORMATION TECHNOLOGY

Module Code: ITCAA2-33

Group: 11

Lecturer's Name: LELISA QOBOSHIYANE

Indicate	Yes	No
Plagiarism report attached	X	

Declaration:

I declare that this assessment is my own original work except for source material explicitly acknowledged. I also declare that this assessment or any other of my original work related to it has not been previously, or is not being simultaneously, submitted for this or any other course. I am aware of the AI policy and acknowledge that I have not used any AI technology to generate or manipulate data, other than as permitted by the assessment instructions. I also declare that I am aware of the Institution's policy and regulations on honesty in academic work as set out in the Conditions of Enrolment, and of the disciplinary guidelines applicable to breaches of such policy and regulations.

			% Participated
1	Student Full Name	Donatella Selemela	100
	Student Number	EDUV4854917	
	Contact Number	+27 66 242 2885	
	Signature	D.S.	
2	Student Full Name	Harun Ulvi Denli	100
	Student Number	EDUV4980065	
	Contact Number	+27 64 379 0194	
	Signature	H.D.	
3	Student Full Name	Makwae Motshoane	100
	Student Number	MD.2023.H0L0Y8	
	Contact Number	+27 78 707 4823	
	Signature	M.K.	
4	Student Full Name	Noor Dyer	100
	Student Number	BE.2023.B7L5Y2	
	Contact Number	+27 64 908 9580	
	Signature	N.D.	
5	Student Full Name	Phahleng Motlapema	100
	Student Number	MD.2022.S8S6K9	
	Contact Number	+27 60 816 9594	
	Signature	P.M.	
6	Student Full Name	Qhayiya Luvuyo Matshikiza	100
	Student Number	EDUV4965447	
	Contact Number	+27 76 239 8354	
	Signature	Q.L.M.	

Lecturer's Comments:

Marks Awarded:	%

Signature	Date

AI DECLARATION

I carefully read the assignment instructions, and the extent to which AI may be used for the assignment. **Yes**
I used the following AI system(s)/tool(s): **MyBib, Grammarly**
I used it for the following: **To generate our reference list. All work was completed using these references.**
If I quoted or paraphrased an AI output, I have referenced the relevant tool, version, and the date I used the tool. **Yes**
I still consider this work my own. (i.e., I have not outsourced the final product, or significant portions of it, to AI tools/systems). **Yes**
If required, I can defend my argument/perspective, explain my choices and approach, and can show that I am knowledgeable about the details of my work. **Yes**

2025

ITCAA2-33

INITIAL SUMMATIVE ASSESSMENT (GROUP PROJECT)

GROUP 11

ABSTRACT

This project focused on designing and implementing a smart irrigation system to address South Africa's growing water scarcity and the inefficiencies of traditional irrigation practices. Guided by the objectives we defined in Part 1; the system was built using an Arduino Uno R3 and a Raspberry Pi 3B+. It combines automated soil moisture detection, pump activation, timed irrigation cycles, servo indication, LCD feedback, intrusion detection and maintenance alerts. The design process required defining functional and non-functional requirements, selecting appropriate components. Furthermore, design required developing both hardware and software, making use of multitasking on the Arduino and multithreading on the Raspberry Pi. Testing confirmed that all specifications (Q2.1–Q2.8) were met:

- Pumps responded accurately to simulated soil conditions
- Zones operated independently
- The timer prevented overwatering
- Intrusion detection worked with minimal false positives
- The maintenance counter performed as intended.

Challenges such as power stability, sensor calibration and synchronisation of multithreaded tasks were faced. These challenges were overcome through dual power supplies, calibration mapping and thread-safe programming. Performance evaluation showed that the system fulfilled its goals of water-use efficiency, automation and security. Stress testing demonstrated long-term reliability. In conclusion, the system successfully applied computer architecture concepts to a real-world problem and future enhancements like higher-accuracy sensors, solar power and user-friendly dashboards could further our system's capabilities and adoption in precision agriculture.

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Background

The Merriam-Webster dictionary defines Agriculture as, “the science, art, or practice of cultivating the soil, producing crops, and raising livestock and in varying degrees the preparation and marketing of the resulting products.” Beyond its definition, Agriculture forms a pillar within society for food security and economic stability. This industry is facing rising pressure due to factors like rapid growth of the global population and limited natural resources. Traditional agricultural practices, with emphasis on irrigation, are growing inefficient and unsustainable. To support these claims, part of the executive summary from a report approved by the Water Research Commission and Department of Agriculture, Land Reform and Rural Development was extracted below.

Agriculture accounts for 70% of global freshwater withdrawals worldwide (FAO, 2017, cited in Mutema et al., 2023) with most of the water being used for crop production (Morison et al., 2008, cited in Mutema et al., 2023). Water scarcity has escalated into a global environmental challenge (Srinivasan et al., 2012, cited in Mutema et al., 2023) due to, among many factors, wastages in agriculture, growing demand from cities and industries, climate change and the need to leave enough to sustain ecosystems (Sharma et al., 2015, cited in Mutema et al., 2023). In South Africa, water scarcity has gained a spotlight position with a deficit of 17% predicted by 2030 (WWF, 2017, cited in Mutema et al., 2023). The country is drifting to solutions such as water-shedding as coping strategy for water scarcity. South Africa (SA) is a semi-arid water stressed country whose long-term annual precipitation averages 480 megametres/yr (Dennis and Dennis, 2012, cited in Mutema et al., 2023), which is much lower than the global average of 860 megametres/yr. Irrigated agriculture in the country accounts for at least 62% of the national water demand (DWAF, 2004; SSA, 2010, cited in Mutema et al., 2023). Ironically, agriculture is the least water use efficient sector with reported wastages of up to 45% (DWA, 2013, cited in Mutema et al., 2023), which cannot be accepted (DWA, 2013, cited in Mutema et al., 2023) for a semi-arid country. With 10% of agricultural land in the country being irrigated and consuming 10 221 cubic megametres per year by 2015 (Van Niekerk et al., 2018, cited in Mutema et al., 2023), 4 600 cubic megametres of the water were wasted. The country is striving to increase irrigated land by more than 50%, which can hardly be achievable without increasing water use efficiency (WUE) in the sector (Mutema et al., 2023).

Against this backdrop, the world, and in scope with this project, South Africa is in desperate need of sustainable and intelligent agricultural practices. More importantly, it needs smarter irrigation solutions, to ensure long-term economic growth and environmental stability of the country. The use of smart irrigation systems leveraging IoT, microprocessor-based digital controllers and real-time sensor monitoring offers a transformative solution. Smart irrigation systems enable precise, data focused water management to minimise water wastages and support sustainable agricultural practices. This project uses these principles as a framework to design and implement a smart irrigation system that addresses South Africa’s water efficiency challenges, conserving resources while improving crop productivity. Additionally, this project showcases the practical application of computer architecture concepts within the context of global issues.

1.2 Problem Statement

Traditional irrigation systems in South Africa manage water inefficiently. They do not account for soil and/or environmental conditions, resulting in wastage of up to 45% of available water. Coupled with South Africa's semi-arid climate and growing water deficit, irrigation inefficiencies pose a severe threat to our food security and agricultural sustainability. The shortage of affordable and automated irrigation solutions emphasises the need for a smart irrigation system that leverages IoT. Such systems will solve the problem of water optimisation to reduce losses and ensure reliable crop production.

1.3 Project Objectives

The main objective of this project is to design, develop and test a smart irrigation system that improves water-use efficiency through fully automated, semi-automated and manual processes. Specifically, the system must achieve the following:

- Accurately detect soil moisture levels using sensors, ensuring real-time identification of dry areas.
- Automate control of sprinkler heads linked to each sensor, watering only when and where required.
- Indicate pump operation using a servo motor to regulate water flow with three distinct speed states.
- Display the areas requiring watering for real-time tracking.
- Limit watering duration to a maximum of 30 seconds or when soil is moist using a switch.
- Display remaining watering duration in real-time.
- Utilise intrusion detection mechanisms and activate an alert if a person/animal enters the irrigation area during watering.
- Perform multithreading to enhance overall system performance by running critical tasks at the same time.
- Provide maintenance alerts after 30 completed irrigation cycles.

1.4 Critical Analysis of IoT Benefits in Irrigation

The paper "An overview of smart irrigation systems using IoT" (Obaideen et al., 2022) explores to what extent Internet of Things (IoT) technology can reform irrigation systems employed in agriculture. This critical analysis evaluates five benefits of implementing and deploying an IoT monitoring system in irrigation, interpreted from the findings in the paper. These benefits are vital to positively impacting semi-arid regions like South Africa.

<u>Water Conservation</u>

Soil moisture and weather sensors monitor real-time environmental conditions and trigger irrigation only when/where required. This conditional approach of irrigation by requirement prevents excessive watering of the soil, saving substantial amounts of freshwater. In South Africa, where irrigated agriculture accounts for more than 60% of national water demand but loses as much as 45% of consumption (Mutema et al., 2023), IoT systems directly address unsustainable practice and enhance our national water security for the future.

Optimised Crop Health and Productivity

IoT networks give us on-demand information regarding soil conditions, allowing us to irrigate accordingly. Through the prevention of flooding and/or crop drought, IoT aids crop resistance, yield and overall growth. This positively impacts South Africa's semi-arid agriculture, where unpredictable rainfall and inefficient water-use threaten our agricultural output.

Cost Efficiency and Reduced Labour

Automated irrigation reduces human labour requirements, while precise irrigation reduces energy and input expenditure. Reduced operational costs would be advantageous to both small-scale and commercial farmers. In South Africa where there is a desire to expand agriculture, IoT offers a way to increase irrigation without having labour/expenditure rise to exponential amounts.

Environmental Sustainability

IoT contributes to more efficient use of water and energy, supporting the United Nations' Sustainable Development Goal 6. IoT minimises agriculture's carbon footprint by conserving energy and surrounding ecosystems. For South Africa, where the National Water Act (No. 36 of 1998) prioritises water-use efficiency (Mutema et al., 2023), IoT represents a viable method for balancing policy and environmental potential.

Scalability

IoT architecture is modular and compatible with wireless networks, cloud platforms and Artificial Intelligence. The flexibility of IoT enables both small and large agricultural hubs to adopt smart irrigation monitoring systems. However, IoT scalability in South Africa depends on overcoming restraints, i.e. limited rural internet access and technical training.

Though there are evident benefits of IoT monitoring systems in irrigation, its adoption in the case of South Africa, rests upon various factors. Utilisation with existing infrastructure, technical training and initial investment may hinder the speed of adoption in under-developed regions. Maintenance of the system is also an evident impediment. Thus, though the benefits of IoT are favourable, implementation must be cost-effective and suit rural South Africa. Nevertheless, these systems present an unmissable chance to realign our irrigation methodology and resource usage in a sustainable way.

2. SYSTEM REQUIREMENTS AND SPECIFICATIONS

2.1 Functional Requirements

- Users must be able to detect soil dryness and activate sprinklers automatically.
- Users must be allowed to override irrigation cycles through a switch.
- Users must be able to control pump operation at variable speeds depending on the number of areas being watered.
- Users must be able to limit watering periods to a maximum of 30 seconds or manually terminate watering.
- Users must be able to view the remaining number of seconds during watering.
- Users must be able to detect the presence of an animal or person in the irrigation area during watering and be alerted.
- Users must be able to run sensor monitoring, pump control, display updates and intrusion detection simultaneously.
- Users must be notified after 30 irrigation cycles that a system service is required.

Summary of functional requirements: Automated Watering, Manual and Semi-Automatic control, Pump Speed Regulation, Timed Watering, Countdown Display, Intrusion Detection, Multithreading, Maintenance Alert

2.2 Non-functional Requirements

- The system must respond to sensor input and activate sprinklers within 0-1 seconds to prevent over/under watering.
- The system should operate without failure at a consistent rate (between 95%-100%) before the 30-cycle maintenance.
- The system must be simple and intuitive to use, allowing those with limited technical knowledge to operate it.
- The system should be designed with replicability and modularity in mind, i.e. adding another microcontroller and sensors without massive infrastructure changes.
- Safety mechanisms integrated into the system must effectively terminate irrigation to protect the system.
- The system should minimise power consumption where possible.
- The system should be adapted to different agricultural settings (home gardens, small plots, medium/large farms).

Summary of non-functional requirements: High Performance, Reliability, Usability, Scalability, Maintainability, Safety Mechanisms, Efficiency, Portability

2.3 Detailed Project Specifications

1. Soil Moisture Sensing: Three soil moisture sensors to detect whether an area is wet or dry.

Project Brief 2.1: "The irrigation system should have three sensors to detect if an area is dry or not."

2. Sprinkler Activation: Three sprinkler heads, each linked to a soil moisture sensor, will activate automatically when its linked sensor detects a dry area.

Project Brief 2.2: "The irrigation system should have three sprinkler heads, each located close to the sensors described above and each sprinkler head should open automatically if a sensor has detected a dry area next to it."

3. Pump regulation: A servo motor must simulate the irrigation pump, operating at three speed states based on the number of areas being watered. A display must indicate the areas being watered.

Project Brief 2.3: "A servo motor is used to indicate the operation of the pump and should have three states operations indicating the required speed of the pump based on the number of areas being watered. A display should be used to indicate the area being watered."

4. Timed Watering: Irrigation must be limited to a maximum of 30 seconds per cycle, or manual early termination when a switch is closed.

Project Brief 2.4: "The Watering period should be controlled by a switch and should be set to a maximum of 30 seconds. While watering an area, if the switch is closed the watering process should end otherwise, it should ends after 30 seconds."

5. Countdown Display: The display mentioned in 3 must show the number of seconds remaining during each watering cycle.

Project Brief 2.5: "Once the watering process has started, the remaining number of seconds of the watering period should be displayed."

6. Intrusion Detection: A motion sensor must monitor the irrigation areas during operation, if an intrusion is detected watering must pause and an alert action must be triggered.

Project Brief 2.6: "It is important to ensure that a person or animal should not enter an area during watering period as such a sensor should be used to detect an intrusion in the area being watered. An alert action is required to prevent intrusion should be performed in case of a detection."

7. Multithreading: The system must implement multithreading to execute critical tasks concurrently.

Project Brief 2.7: "The irrigation system should operate at high performance by implementing multithreading of tasks."

8. Maintenance Alert: After 30 completed irrigation cycles, the system shall display a notification to prompt inspection and servicing.

Project Brief 2.8: “After 30 times that the irrigation system has watered, maintenance is required, in this case a maintenance sign should be displayed.”

3. SYSTEM DESIGN

3.1 High-level Block Diagram

Please see appendix D, figure 1 for full reference. Our high-level block diagram pictured before demonstrates our completed smart irrigation system's main functional blocks. The "Power" block is comprised of a common ground bus, an external 6 V power supply, breadboards and a breadboard PSU module. This block is responsible for carrying external power from the 6 V power supply, using the breadboard PSU module, to the pumps and relays. It is further responsible for grounding the entire system through a common ground bus and powering the other blocks from the respective microcontrollers ("Compute and Control" block). The "Compute and Control" block is comprised of our Raspberry Pi and Arduino. This block is responsible for processing all sensor inputs, executing irrigation logic, and coordinating system outputs using multitasking. The Arduino manages irrigation tasks such as reading soil moisture sensors, checking rain, light and environmental conditions, driving the relays that control the pumps, updating the servo indicator, and refreshing the LCD display with countdowns, alerts and maintenance prompts. The Raspberry Pi functions as the supervisory controller, running multithreaded processes for intrusion detection (PIR and ultrasonic sensors), sounding the buzzer, pausing irrigation when maintenance is required, maintaining the cycle counter for maintenance, and sending system data to the IoT cloud over Wi-Fi. Both microcontrollers communicate with each other over USB. The "Sensors (Arduino I/O)" and "Sensors (Raspberry Pi I/O)" blocks represent our sensors connected to each microcontroller that read data from the external environment and send it to the microcontroller for processing over GPIO, allowing the system to complete functional requirements and non-functional requirements. The "Actuators and Outputs" block includes three DC pumps (one per zone), the relay modules used to switch them, a servo motor to indicate pump state, the LCD display for countdown and system status, and the passive buzzer for intrusion alerts. This block responds to actions from the "Compute and Control" block, making outputs from processing visible to the user in a measurable manner. Finally, the "Cloud" block represents the Raspberry Pi's link to the IoT cloud over Wi-Fi. This block ensures that users can view system performance and receive intrusion or maintenance notifications beyond physical hardware.

3.2 Hardware Design

3.2.1 Component Selection

We have chosen to use both a Raspberry Pi 3 B+ and an Arduino Uno R3 as our microcontrollers. The Arduino Uno R3 excels at I/O processing, more so with complex systems, than the Raspberry Pi. However, the Uno R3 does not have IoT capabilities, as it cannot connect to the internet, nor does it have a CPU capable of multithreading. The Raspberry Pi fills the gaps of the Arduino to ensure effective and responsive system functionality. For sensors, capacitive soil moisture sensors were chosen over resistive soil moisture sensors, as they do not suffer corrosion over time. On the actuator side, three 3.5 V DC water pumps were selected to provide zone-based irrigation. These are controlled by a 1-channel relay and a 2-channel relay module, chosen for their ability to handle the pumps' current requirements while offering electrical isolation from the control logic. A servo motor was added as

a physical indicator of active pump states, while an I²C 16×2 LCD was selected for its ability to display zone information, countdown timers, and maintenance alerts in real time. A PIR (passive infrared) sensor was chosen to detect motion, coupled with an ultrasonic sensor limited to 150 cm to confirm object proximity. This dual-sensor approach reduces false positives and ensures accurate intrusion detection. The passive buzzer provides a low-power alert mechanism triggered by the Raspberry Pi in response to intrusion events. A DHT11 sensor was selected for measuring temperature and humidity due to its low cost, reliability and ease of integration with Arduino libraries. A rain sensor was incorporated to prevent unnecessary watering during rainfall and an LDR (light-dependent resistor) was added to help the system avoid irrigation under low-light or high-evaporation conditions. For power, a 6 V external battery pack connected to a breadboard PSU was chosen to supply the pumps and relays, and a 5 V / 2.5 A power bank was selected to supply stable current to the Raspberry Pi and Arduino. This dual power supply approach ensures the pumps' load does not interfere with the microcontrollers' operation. A common ground bus ties all components together, maintaining reliable signal references across the system.

3.2.2 Circuit Diagram

Please see appendix D, figure 2 for full diagram.

3.3 Software Design

3.3.1 Flowchart of the entire system

Please see appendix D, figure 3 for full diagram.

3.3.2 Multithreading Implementation

In our system, we made deliberate use of different concurrency approaches to ensure responsive and reliable performance across all processes. On the Raspberry Pi 3 B+, we implemented multithreading to handle intrusion detection and system supervision. Separate threads were assigned to the ultrasonic distance sensor, the PIR motion sensor and the buzzer control, allowing each of these processes to run independently and in parallel without blocking one another. By doing so, the Pi could detect motion, confirm object distance and trigger alerts simultaneously; while also leaving capacity for publishing status updates to the cloud (VNC over wifi). On the Arduino Uno R3, we applied multitasking using non-blocking timers and interval checks within the main loop. Instead of relying on delays, sensor reads, pump updates, LCD refreshes, and DHT measurements were scheduled at different time intervals. This approach allowed the Arduino to interleave tasks efficiently, so soil moisture, rain and light readings could update at their own frequencies while pumps and countdown timers continued to operate without interruption. Between the Raspberry Pi and the Arduino, we introduced multiprocessing through inter-device communication. The Pi runs as a separate process, connected via serial or I²C to the Arduino, and this boundary enforces a clean separation of responsibilities. The Arduino focuses on irrigation logic and outputs, while the Pi supervises intrusion detection, cycle counting and IoT connectivity. This multiprocessing layer ensures that heavy tasks on the Pi do not interfere with time critical I/O tasks on the Arduino, keeping the system high performing.

4. SOFTWARE IMPLEMENTATION

4.1 Developed Code

Please see Appendix C for full developed code on both the Arduino and Raspberry Pi.

4.2 Development Environment

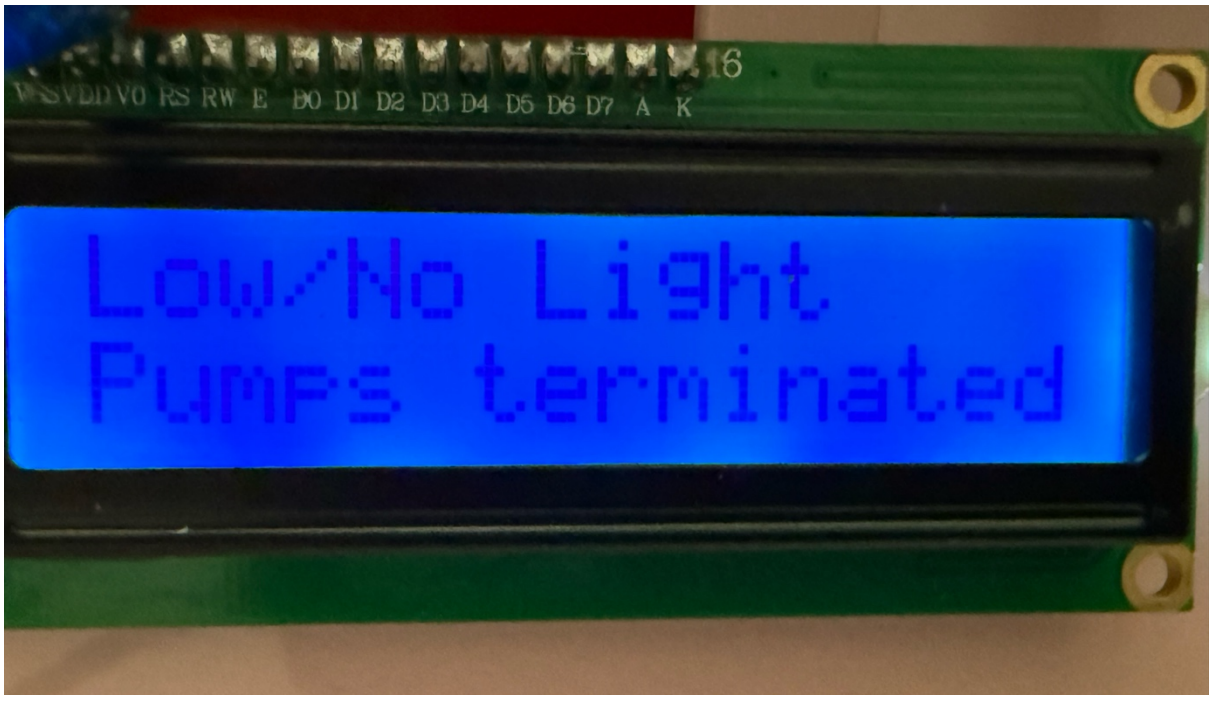
The Arduino Uno R3 was programmed in the Arduino IDE using the avr-gcc compiler. Key libraries included the Adafruit DHT sensor library for temperature and humidity, the Servo library for pump indication and the LiquidCrystal_I2C library for LCD output. The Raspberry Pi 3B+ code was developed in C++ using g++ with the pigpio library for GPIO handling and the standard thread library for multithreading. This allowed ultrasonic sensing, PIR motion detection and buzzer control to run in parallel while remaining responsive.

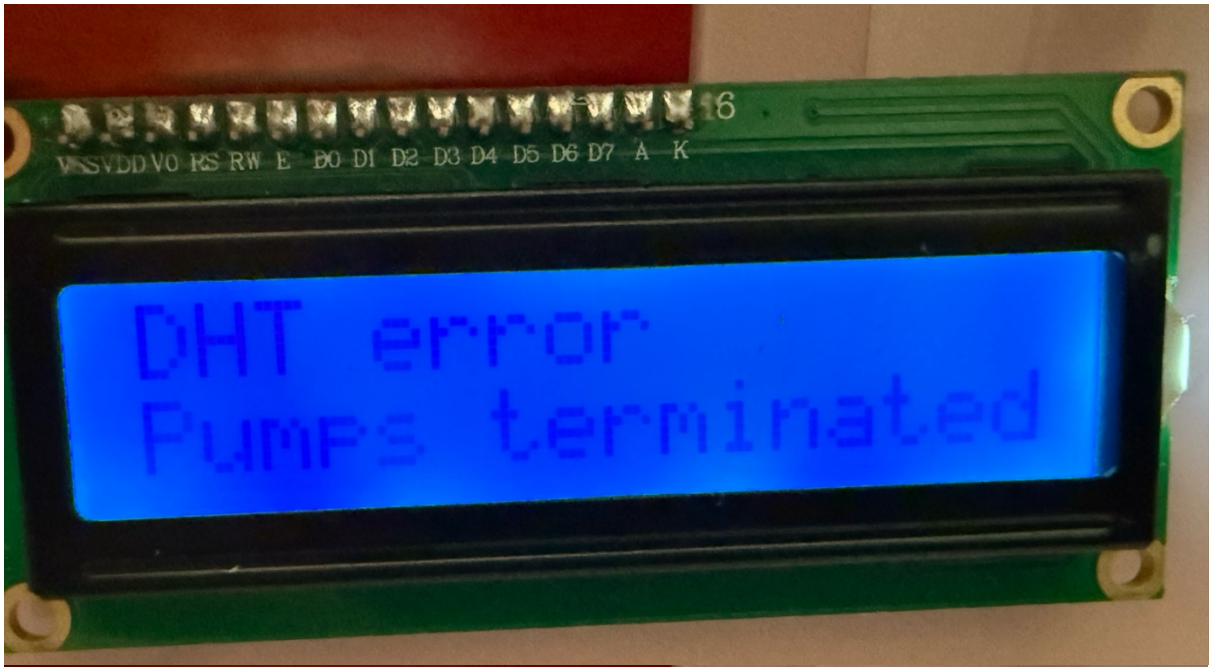
5. SOFTWARE IMPLEMENTATION

5.1 Simulation/Testing Methodology

We followed a staged simulation and testing methodology to ensure our smart irrigation system produced reliable results. Our process began with individual component testing on breadboards. Each sensor and actuator was validated using simple test sketches in the Arduino IDE or standalone scripts on the Raspberry Pi. This confirmed that they produced expected outputs before integration. Next, module-level testing was performed, where related components were grouped together and tested as functional subsystems. For example, the soil moisture sensors were connected to the Arduino and used to trigger the relay-driven pumps. This stage ensured that control decisions matched sensor conditions and that actuators responded accordingly. Following this, we performed integration testing between the Arduino and Raspberry Pi. Communication was simulated over serial links to confirm that intrusion events detected by the Pi successfully triggered the Arduino's safety stop logic, and that irrigation data from the Arduino could be sent to the Pi without errors. This validated our multiprocessing design. To simulate real-world conditions, we carried out scenario testing. Dry soil samples were used to confirm pump activation below the VPD threshold, while wet soil verified shutdown above 60%. The rain sensor was tested by applying water directly to ensure irrigation was prevented, and light levels were varied over the LDR to simulate daytime and nighttime conditions. Intrusion events were simulated by moving objects at varying distances in front of the PIR and ultrasonic sensors to test both detection and false positive rejection. Finally, system stress testing was performed by running multiple irrigation cycles in sequence. This confirmed the countdown timer on the LCD, the servo indication of pump activity and the triggering of the maintenance alert after 30 cycles. Power stability was also tested by running pumps on the 6 V supply while ensuring the 5 V power bank consistently powered the Raspberry Pi and Arduino.

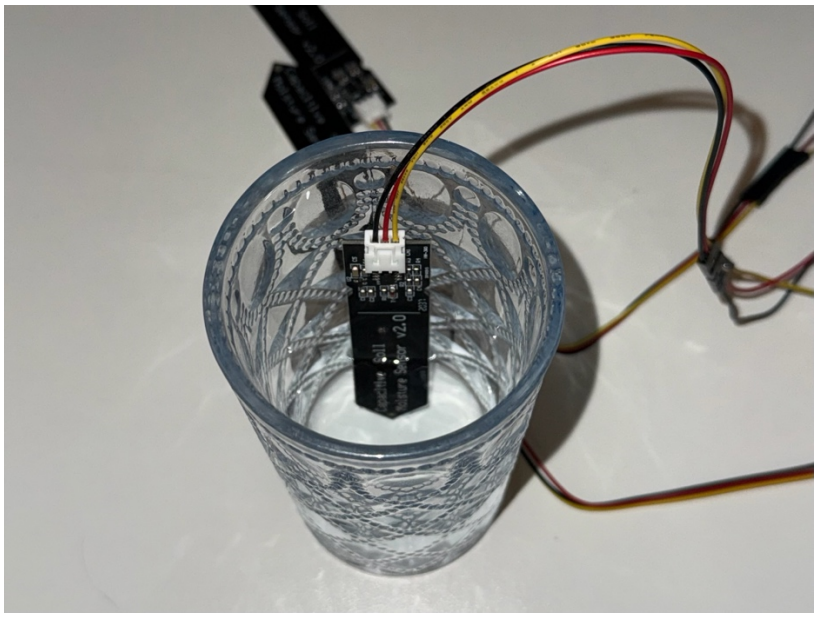
5.2 Results




VSS VDD VO RS RW E D0 D1 D2 D3 D4 D5 D6 D7 A K
16
DHT error
Pumps terminated


VDD VO RS RW E D0 D1 D2 D3 D4 D5 D6 D7 A K
16
Rain detected
Remove System!

Images above show successful Termination during stress testing

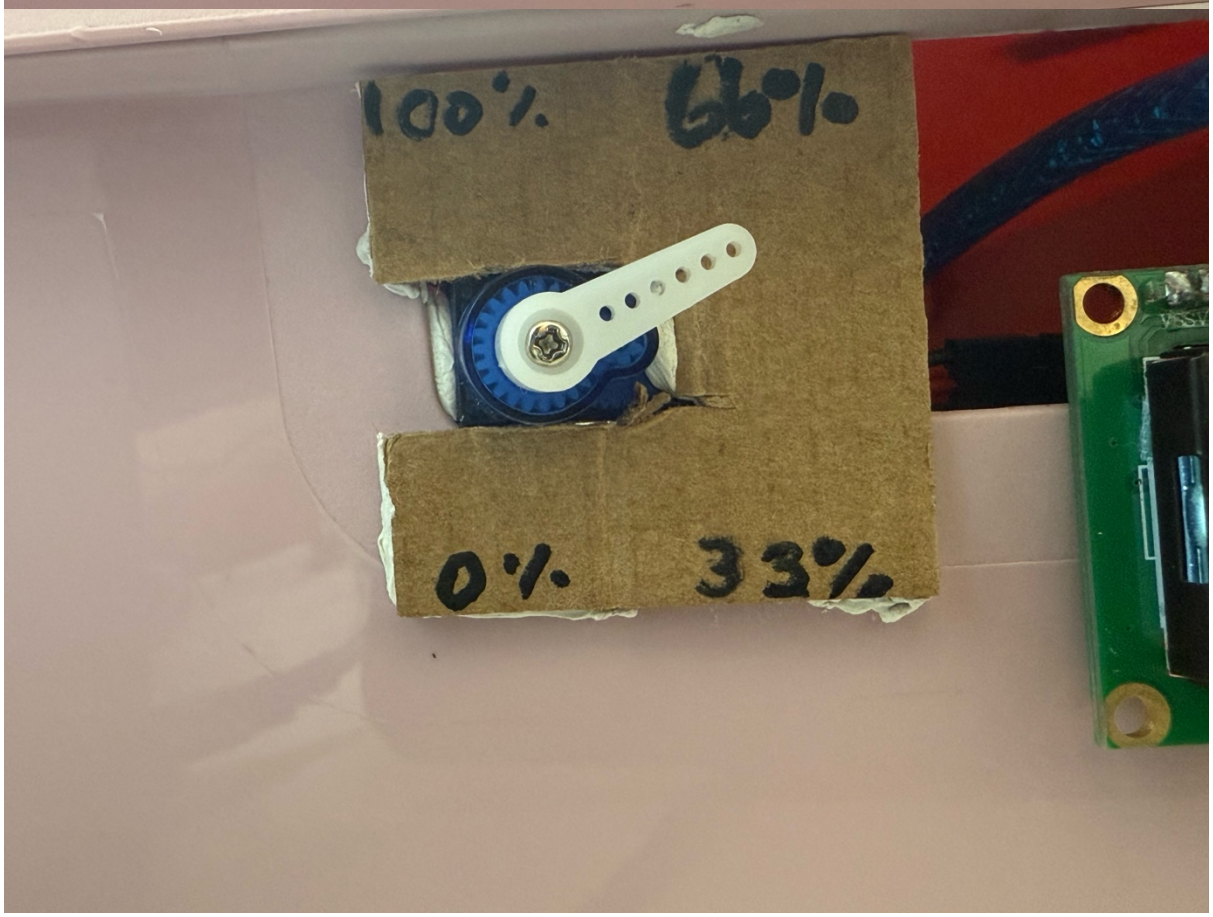



All areas
Time left: 29s

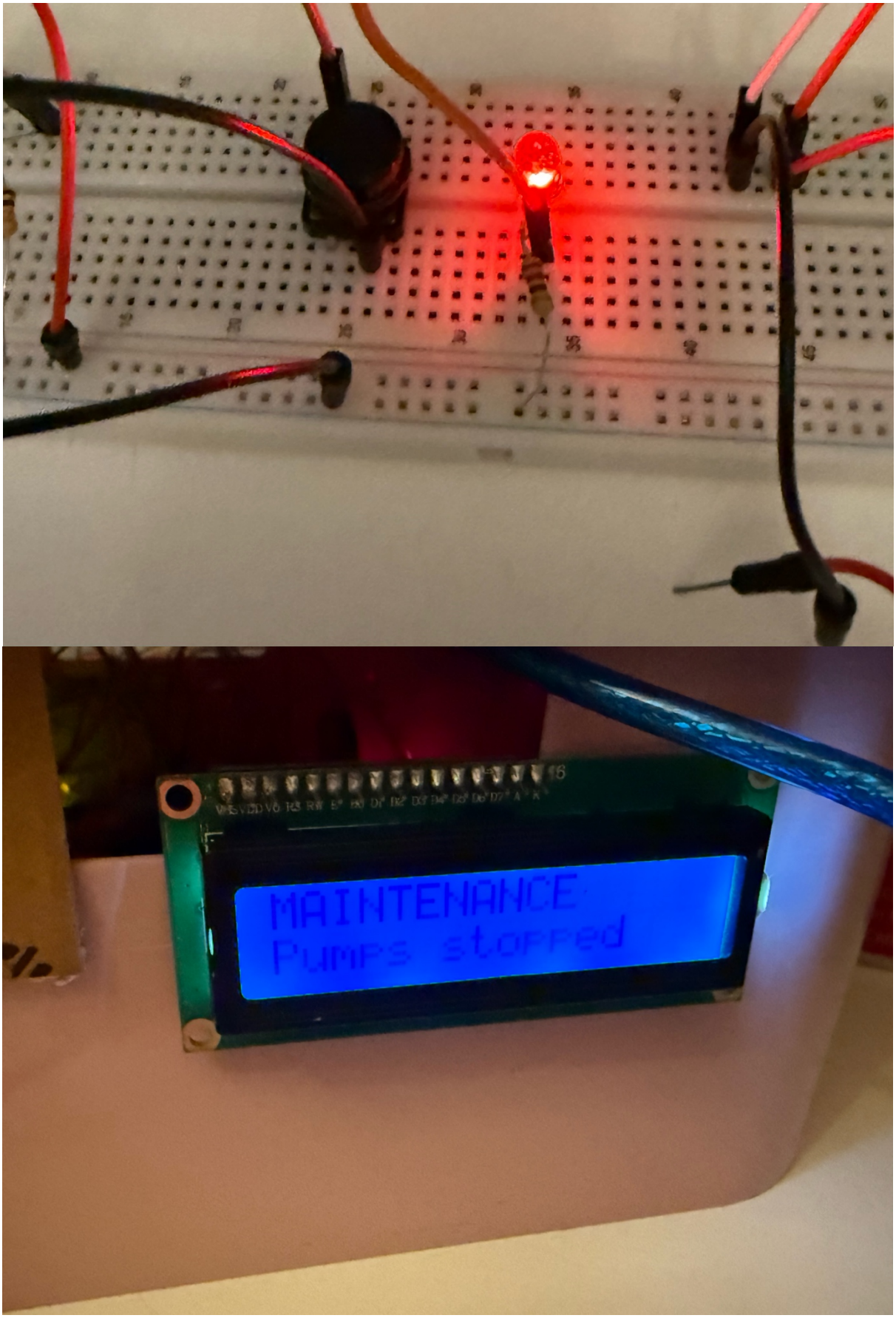

Area 2.3
Time left: 2s


Area 3
Time left: 6s

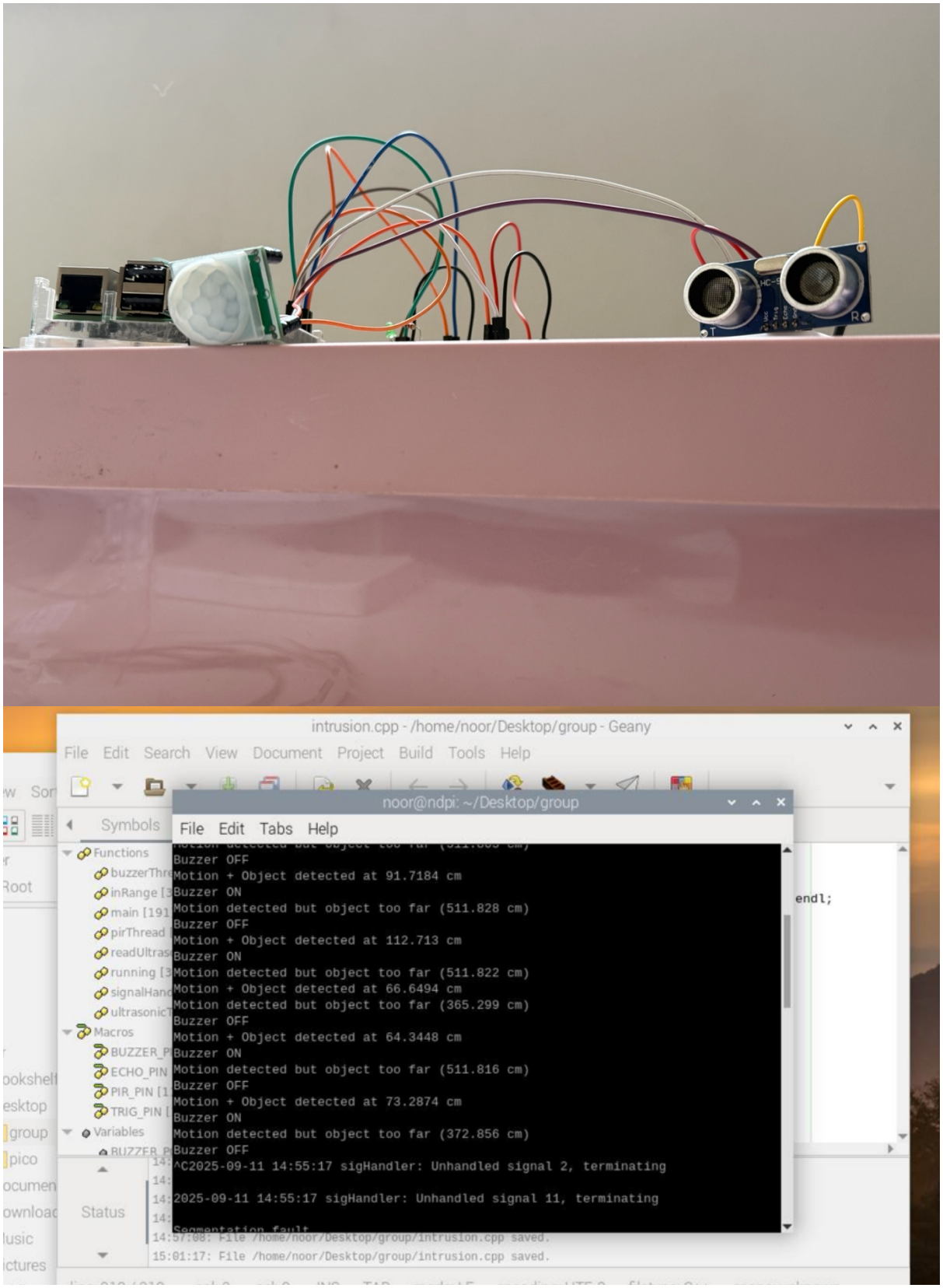

100%
66%
0%
33%
VSS VDD VO RS RW E D0 D1 D2 D3 D4 D5 D6 D7 A K
All areas
Time left: 9s


100%
66%
0%
33%

The images above show successful runs during main functional testing



The images above show successful termination after 30 cycles (completed or terminated)



The images above shows the intrusion detection setup with an LED in place of the buzzer to prevent noise and show for image. Proof of multithreading included as well.

The testing phase confirmed that the smart irrigation system met its design requirements with results documented. The first set of images show the system successfully detecting low soil moisture levels. When readings fell below the VPD threshold, the Arduino activated corresponding pumps via the relays. Once the moisture level rose above the set threshold of 60% (average good moisture for plants), the pump automatically switched off. This confirmed that automated irrigation control logic worked reliably, responding directly to real soil conditions. Testing further confirmed that each of the three zones operated independently. The images demonstrate that when Zone 1 soil was dry, only Pump 1 activated and when multiple zones required water, multiple pumps operated simultaneously. This zone-based approach reduced unnecessary water consumption, as irrigation was limited to the areas that needed it. The servo motor provided a physical representation of irrigation intensity. As shown in the test images, the servo rotated incrementally depending on the number of active pumps. The angles are 0° when idle, 60° for one active pump, 120° for two and 180° when all three were running. This simple visual mechanism offered a quick way to monitor pump activity without checking the LCD. Stress testing demonstrated that pumps terminated reliably after the maximum 30-second watering period. The LCD countdown, captured in the images, shows the timer decrementing to zero before irrigation stopped. This safeguard ensured consistent cycle times and prevented overwatering, even if the soil remained below the moisture threshold. The LCD display provided real-time feedback throughout testing. During irrigation, it showed the active zone(s) on the first line and the remaining countdown time on the second line. When no pumps were active, the screen displayed “No active pumps.” The images confirm that the display updated smoothly, providing accurate status information and alert messages without noticeable lag or flicker. The intrusion detection subsystem was validated using a PIR sensor combined with an ultrasonic distance sensor. To avoid excessive noise during testing, the buzzer was temporarily replaced with an LED indicator. The images show the LED illuminating when both motion and object proximity within 150 cm were detected. This confirmed intrusion detection logic worked well by reducing false positives through distance and motion conditions. Multithreading intrusion on the Raspberry Pi improved performance by allowing ultrasonic distance sensing, PIR detection and buzzer control to run simultaneously on separate threads. This meant the system could detect motion, confirm object distance and trigger alerts in real time without blocking or delays. The captured code snippets demonstrate the use of the C++ thread library, while the testing images confirm responsive behaviour under different conditions. Compared to a sequential loop, multithreading reduced latency, avoided missed events and ensured continuous logging to the console; while still reacting instantly to intrusions. Long-duration testing confirmed that the maintenance counter functioned correctly. After 30 completed watering cycles, either due to soil termination or faults, the LCD displayed “Maintenance Required” as shown in the images. The pump states and maintenance flag was stored in a text file for operation during concurrent runs, while the system is powered, showing state persistence. This ensures users are reminded to service the system after set usage during operation. Overall, the results confirm that our smart irrigation system fulfils all functional and non-functional requirements.

Relevant code snippets:

1. *Automated, zone-based irrigation logic — see Appx. D: Zone-based Pump Control (independent activation & thresholds) in Arduino code.*

2. *Start/Stop thresholds (≤40% on, ≥60% off) — see Appx. D: Threshold logic per zone (Arduino).*

3. *30-second watering safeguard & countdown — see Appx. D: 30-Second Safeguard (countdown & hard stop) (Arduino).*

4. *Independent zone operation (1–3 pumps simultaneously) — see Appx. D: Zone-based Pump Control and Servo Indicator (Arduino).*

5. *Servo visualization of irrigation intensity (0°/60°/120°/180°) — see Appx. D: Servo Indicator (visual intensity by active pumps) (Arduino).*

6. *LCD real-time status (active zones + timer / alerts) — see Appx. D: LCD Status (active zones + countdown; smooth updates) (Arduino).*

7. *Noise-reduced console logging — see Appx. D: Significant-Change Serial Logging (Arduino).*

8. *Rain / low-light / DHT fault interlocks (pump termination) — see Appx. D: stop conditions inside Zone-based Pump Control and alert branches in LCD Status (Arduino).*

9. *Cycle counting for maintenance (increments on completed stops) — see Appx. D: Stop pump (counts a completed cycle) (Arduino).*

10. *Intrusion detection (PIR + ultrasonic ≤150 cm) — see Appx. D: Thread: motionThread (Pi C++).*

11. *Ultrasonic smoothing & print threshold (moving average) — see Appx. D: Thread: ultrasonicThread (Pi C++).*

12. *Non-blocking alert tone control — see Appx. D: Thread: buzzerThread (Pi C++).*

13. *Real-time serial ingest from Arduino (parses “Pump Cycles”) — see Appx. D: Thread: serialReaderThread (Pi C++).*

14. *Maintenance watchdog (threshold = 30 cycles) — see Appx. D: Thread: watchdogThread (Pi C++).*

15. *State persistence to file (for concurrent runs) — see Appx. D: State Persistence (simple text file log) (Pi C++).*

16. *Manual reset & maintenance toggle (button + serial commands) — see Appx. D: Button callback (RESET:0,0,0 / MAINT:OFF) and Arduino Serial handling (MAINT:ON/OFF, RESET).*

17. *Multithreading benefits (no blocking between sensing/alerts/IO) — see Appx. D: combined ultrasonicThread, motionThread, buzzerThread, serialReaderThread, watchdogThread (Pi C++).*

18. *System readiness & UX messages (LCD “No active pumps”, countdown, alerts) — see Appx. D: LCD Status (Arduino).*

19. *Zone independence verified (Pump 1 only when Zone 1 dry; multiple pumps when multiple zones dry) — see Appx. D: Zone-based Pump Control (Arduino) and Significant-Change Serial Logging.*

20. *Fault handling & safe termination (DHT error, rain, low light) — see Appx. D: stop conditions in Zone-based Pump Control and alert branches in LCD Status (Arduino).*

5.3 Challenges and solutions

During the design and implementation of the smart irrigation system, several challenges were encountered. The first major challenge was power management. Pumps and relays require a higher current than the microcontrollers could provide with all the sensors. This initially caused instability. We solved power problems by splitting the power supply into two sources. Next, a common ground bus was established to ensure stable reference levels to the microcontrollers across all their respective components. Lastly, when more than 2 pumps were on at a time (unlikely to happen in real-life but still possible), the noise generated by the relays turning on and off disrupted functionality. The solution to this issue is to add diodes and capacitors to the pumps, but we did not have these during testing due to time constraints.

Another challenge was analog sensor calibration, particularly for our capacitive soil moisture sensors. Raw analog readings varies across the three due to manufacturing defects. We resolved calibration for all sensors by taking baseline readings and mapping their individual ranges to percentage values. This ensured our sensors behaved consistently across zones.

Timing and multitasking on the Arduino was our next challenge. Early versions of the code relied on delays which blocked other functions. This issue was overcome by implementing non-blocking multitasking using millis() timers and separating the loop into helper functions. Each task was assigned an interval, allowing them to run in parallel without interfering with each other. On the Raspberry Pi, implementing multithreading introduced synchronisation issues between the PIR, Arduino and ultrasonic sensors. At first, threads were not coordinated which lead to missed detections or false positives/negatives. This was addressed by introducing shared atomic variables and mutex locks, ultimately ensuring consistent data exchange across threads.

Finally, testing the intrusion detection system presented practical challenges. Continuous buzzer activation during experiments was disruptive, making it difficult to observe results. To solve this an LED was temporarily used in place of the buzzer as a visual indicator, allowing intrusion events to be validated without noise. Once functionality was confirmed, the buzzer was reintroduced for final testing. Through these solutions the system was able to achieve stable power delivery, accurate sensing, efficient multitasking and reliable multithreaded intrusion detection, effectively meeting all project specifications.

5.4 Performance evaluation

The smart irrigation system was critically evaluated against the initial requirements set out in Q2.1–Q2.8, and the results demonstrate that it met expectations across all major functions. The soil moisture detection and pump control (Q2.1) worked reliably, with pumps activating below VPD thresholds and stopping above 60%. The zone-based design (Q2.2) successfully provided independent control of three irrigation areas, ensuring water was directed only where needed. This directly addressed the project's efficiency goals by avoiding unnecessary irrigation. The servo motor indicator (Q2.3) offered a clear physical representation of system activity, rotating proportionally to the number of pumps running. The watering timer

(Q2.4) performed consistently, limiting cycles to 30 seconds and preventing overwatering. Together, these features ensured controlled irrigation and gave visible confirmation of system behaviour. The LCD module (Q2.5) delivered real-time feedback, displaying zone status, countdown timers and maintenance alerts. This allowed users to monitor the system without requiring a serial connection or cloud access. The intrusion detection subsystem (Q2.6), combining PIR and ultrasonic sensing, proved accurate in detecting nearby motion with false positives reduced through distance validation. The buzzer alert was effective and the substitution with an LED during testing confirmed the logic without unnecessary disruption. The use of multithreading on the Raspberry Pi (Q2.7) improved responsiveness significantly. By running ultrasonic sensing, PIR detection and buzzer control in parallel threads, the system was able to detect and respond to events in real time while still publishing logs to the cloud. Without multithreading, sequential processing would have introduced latency and risked missed detections. Furthermore, the inclusion of multitasking on the Arduino ensured that a sequential loop did not block any sensor data, common with a single loop and delays. Finally, the maintenance counter (Q2.8) met the long-term reliability requirement with the system correctly displaying a “Maintenance Required” alert after 30 cycles. This feature strengthens sustainability by prompting timely servicing.

Overall, the system met or exceeded all functional requirements. It performed reliably under normal operation, adapted correctly to environmental conditions and maintained stability during stress testing. While improvements could be made to sensor accuracy and long-term robustness in outdoor environments, the prototype achieved its design objectives of water efficiency, automation, and security.

6. CONCLUSION

The smart irrigation system we built, combining the Arduino Uno R3 for irrigation control and the Raspberry Pi 3B+ for supervision, successfully met all our requirements. It addressed the challenges highlighted in 1 by responding to real soil conditions, managing water use through timed cycles and giving users both visual and physical feedback. The tests showed that the system runs reliably under normal and stress conditions. These results prove that our system addresses South Africa's water efficiency challenges, conserving resources while improving crop productivity. Additionally, this project showcases the practical application of computer architecture concepts within the context of global issues.

7. REFERENCES

Adafruit (2024). *adafruit/DHT-sensor-library*. [online] GitHub. Available at: https://github.com/adafruit/DHT-sensor-library?tab=readme-ov-file [Accessed 2 Sep. 2025].

Aggie Horticulture (2023). *Light, Temperature and Humidity | Ornamental Production*. [online] Tamu.edu. Available at: https://aggie-horticulture.tamu.edu/ornamental/a-reference-guide-to-plant-care-handling-and-merchandising/light-temperature-and-humidity/ [Accessed 2 Sep. 2025].

Arduino (2024). *Arduino® UNO R3*. [online] Available at: https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf [Accessed 1 Sep. 2025].

Arduino Get Started (2025). *Arduino - Rain Sensor | Arduino Tutorial*. [online] Arduino Getting Started. Available at: https://arduinogetstarted.com/tutorials/arduino-rain-sensor [Accessed 2 Sep. 2025].

Baverstock, T. (2019). *Fritzing Tutorial - A Beginners Guide to Making Circuit & Wiring Diagrams*. *YouTube*. Available at: https://www.youtube.com/watch?v=-saXw1EipX0 [Accessed 20 Aug. 2025].

Dunn, C. (2020). *DIY Raspberry Pi Arduino Plant Watering System*. [online] www.youtube.com. Available at: https://www.youtube.com/watch?v=DOaDnYj3vfI [Accessed 24 Aug. 2025].

Flaura - Smart Plant Pot (2022). *Capacitive Soil Moisture Sensors don't work correctly + Fix for v2.0 v1.2 Arduino ESP32 Raspberry Pi*. [online] www.youtube.com. Available at: https://www.youtube.com/watch?v=IGP38bz-K48 [Accessed 28 Aug. 2025].

friendlywire (2021). *SG90 9 g Micro Servo*. [online] Available at: https://www.friendlywire.com/projects/ne555-servo-safe/SG90-datasheet.pdf [Accessed 4 Sep. 2025].

Grant, B.L. (2025). *When Is The Best Time To Water Plants? Expert Guide To Keep Plants Happy Even In The Heat*. [online] Gardening Know How. Available at: https://www.gardeningknowhow.com/garden-how-to/watering/best-time-to-water-plants [Accessed 2 Sep. 2025].

Grow, P. (2022). *The Ultimate Vapor Pressure Deficit (VPD) Guide*. [online] Pulse Grow. Available at: https://pulsegrow.com/blogs/learn/vpd?srsltid=AfmBOoq3nw92cRzUqqO0cxdJfTxGo2zdQF8w_Om-sebhOoPUZAztcTvo#vpth [Accessed 4 Sep. 2025].

Handson Technology (2016a). *Handson Technology*. [online] Available at: https://www.handsontec.com/dataspecs/mb102-ps.pdf [Accessed 3 Sep. 2025].

Handson Technology (2016b). *Handson Technology User Guide 2 Channel 5V Optical Isolated Relay Module*. [online] Available at: https://www.handsontec.com/dataspecs/2Ch-relay.pdf [Accessed 3 Sep. 2025].

Handson Technology (2016c). *Handson Technology User Guide HC-SR501 Passive Infrared (PIR) Motion Sensor*. [online] Available at: https://www.handsontec.com/dataspecs/SR501%20Motion%20Sensor.pdf.

Handson Technology (2017). *Handson Technology User Guide HC-SR04 Ultrasonic Sensor Module User Guide User Guide: Ultrasonic Sensor V2.0*. [online] Available at: https://www.handsontec.com/dataspecs/HC-SR04-Ultrasonic.pdf.

Handson Technology (2018). *Handson Technology User Guide I2C Serial Interface 1602 LCD Module*. [online] Available at: https://www.handsontec.com/dataspecs/module/I2C_1602_LCD.pdf [Accessed 4 Sep. 2025].

Handson Technology (2019). *Handson Technology User Guide*. [online] Available at: https://handsontec.com/dataspecs/relay/1Ch-relay.pdf [Accessed 3 Sep. 2025].

HK Shan Hai Group Limited (2016). *Snow & Raindrops Detection Sensor Module*. [online] Available at: https://www.dessy.ru/include/images/ware/pdf/s/snow_&_raindrops_detection.pdf [Accessed 2 Sep. 2025].

Joy-IT (2024). *KY-018 Photoresistor module KY-018 Photoresistor module Contents*. [online] Available at: https://fabacademy.org/2024/labs/puebla/students/andrea-hortega/assets/images/week11/KY-018-Joy-IT.pdf [Accessed 2 Sep. 2025].

lastminuteengineers (2023). *Interfacing Capacitive Soil Moisture Sensor with Arduino*. [online] Last Minute Engineers. Available at: https://lastminuteengineers.com/capacitive-soil-moisture-sensor-arduino/ [Accessed 1 Sep. 2025].

Mutema, M., Dhavu, K., Vorster, S., Reinders, F., Sivhagi, P., Sekgala, M., Benadé, N. and Benadé, N. (2023). *THE STATE OF IRRIGATION WATER LOSSES AND MEASURES TO IMPROVE WATER USE EFFICIENCY ON SELECTED IRRIGATION SCHEMES*. [online] *Wrcwebsite*. Available at: https://wrcwebsite.azurewebsites.net/wp-content/uploads/mdocs/2970.pdf [Accessed 11 Aug. 2025].

Obaideen, K. (2022). An overview of smart irrigation systems using IoT. *Energy Nexus*, [online] 7(7), p.100124. doi:https://doi.org/10.1016/j.nexus.2022.100124.

OKdo and RS Components (2018). *Raspberry Pi 3B+*. [online] docs.rs-online.com. Available at: https://docs.rs-online.com/a608/A700000007750677.pdf [Accessed 2 Sep. 2025].

plnts (2025). *Temperature and humdity*. [online] PLNTS.com. Available at: https://plnts.com/en/care/doctor/temperature-and-humidity [Accessed 2 Sep. 2025].

Rabbi, B., Chen, Z.-H. and Sethuvenkatraman, S. (2019). Protected Cropping in Warm Climates: A Review of Humidity Control and Cooling Methods. *Energies*, 12(14), p.2737. doi:https://doi.org/10.3390/en12142737.

rcscomponents (2013). *Temperature and humidity sensor KY-015*. [online] Available at: https://www.rcscomponents.kiev.ua/datasheets/ky-015-datasheet.pdf?srsltid=AfmBOoq_3-LtqfUV7rOFgfAo7l3s9cP9zv0rVq1iDrw7yLHveuLZuJIw [Accessed 1 Sep. 2025].

Redmon, M. (2024). *How Moist Should Soil Be? A Soil Moisture Gardening Guide*. [online] WeatherFlow-Tempest, Inc. Available at: https://tempest.earth/resources/how-moist-should-soil-be/ [Accessed 2 Sep. 2025].

Scotty D (2021). 🌧️ *Rain Sensor module: Wiring and Coding Guide for Arduino* 💻🛠️. [online] YouTube. Available at: https://www.youtube.com/watch?v=IKh5x6YurYs [Accessed 2 Sep. 2025].

Shenzhen haiwang sensor co (2015). *HW-101 HW-moisture sensor V1.2 Specification*. [online] Available at: https://www.datocms-assets.com/28969/1662716326-hw-101-hw-moisture-sensor-v1-0.pdf [Accessed 1 Sep. 2025].

SriTu Hobby (2022). *How to make an irrigation monitoring system with Raspberry Pi board | Raspberry Pi Projects*. [online] YouTube. Available at: https://www.youtube.com/watch?v=s_Ecar_yMBM [Accessed 24 Aug. 2025].

Stevens, J., Van Heerden, P. and Laker, M. (2012). *Training material for extension advisors in irrigation water management Volume 2: Technical Learner Guide Part 1: Soil-plant-atmosphere continuum*.

The Robotics Back-End (2019). *Raspberry Pi Arduino Serial Communication - Everything You Need To Know*. [online] The Robotics Back-End. Available at: https://roboticsbackend.com/raspberry-pi-arduino-serial-communication/ [Accessed 2 Sep. 2025].

Workshop, D. (2022). *Water Your Garden with IoT*. [online] DroneBot Workshop. Available at: https://dronebotworkshop.com/soil-moisture/ [Accessed 28 Aug. 2025].

APPENDIX A: COMPONENT DATASHEETS

1. Arduino Uno R3

Arduino (2024). Arduino® UNO R3. Datasheet. Available at: https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf

2. Raspberry Pi 3B+

OKdo and RS Components (2018). Raspberry Pi 3B+. Available at: https://docs.rs-online.com/a608/A700000007750677.pdf

3. Capacitive Soil Moisture Sensor

Shenzhen haiwang sensor co (2015). *HW-101 HW-moisture sensor V1.2 Specification*. Available at: https://www.datocms-assets.com/28969/1662716326-hw-101-hw-moisture-sensor-v1-0.pdf

4. DHT11 Temperature and Humidity Sensor

rcscomponents (2013). *Temperature and humidity sensor KY-015*. Available at: https://www.rcscomponents.kiev.ua/datasheets/ky-015-datasheet.pdf?srsltid=AfmBOoq_3-LtqfUV7rOFgfAo7l3s9cP9zv0rVq1iDrw7yLHveuLZuJIw

5. Rain Sensor Module

HK Shan Hai Group Limited (2016). *Snow & Raindrops Detection Sensor Module*. Available at: https://www.dessy.ru/include/images/ware/pdf/s/snow_&_raindrops_detection.pdf

6. Light Dependent Resistor (LDR)

Joy-IT (2024). *KY-018 Photoresistor Module*. Available at: https://fabacademy.org/2024/labs/puebla/students/andrea-hortega/assets/images/week11/KY-018-Joy-IT.pdf

7. PIR Motion Sensor (HC-SR501)

Handson Technology (2016c). *HC-SR501 Passive Infrared (PIR) Motion Sensor*. Available at: https://www.handsontec.com/dataspecs/SR501%20Motion%20Sensor.pdf

8. Ultrasonic Sensor (HC-SR04)

Handson Technology (2017). *HC-SR04 Ultrasonic Sensor Module User Guide*. Available at: https://www.handsontec.com/dataspecs/HC-SR04-Ultrasonic.pdf

9. Relay Modules (1-Channel & 2-Channel)

Handson Technology (2016b). *2 Channel 5V Optical Isolated Relay Module*. Available at: https://www.handsontec.com/dataspecs/2Ch-relay.pdf

Handson Technology (2019). *1 Channel 5V Relay Module*. Available at: https://handsontec.com/dataspecs/relay/1Ch-relay.pdf

10. Servo Motor (SG90)

friendlywire (2021). *SG90 9 g Micro Servo Datasheet*. Available at: https://www.friendlywire.com/projects/ne555-servo-safe/SG90-datasheet.pdf

11. I²C 16x2 LCD Display Module

Handson Technology (2018). *I2C Serial Interface 1602 LCD Module*. Available at: https://www.handsontec.com/dataspecs/module/I2C_1602_LCD.pdf

12. Passive Buzzer Module

Handson Technology. *Passive Buzzer Module*. Available at: https://www.handsontec.com/dataspecs/module/passive%20buzzer.pdf

APPENDIX B: ADDITIONAL PROCESS IMAGES/CODE

1. Initial outputs requirements before multitasking on the Arduino (2.1-2.6)

```
No rain detected
Soil Moisture (%):
Area 1 - 0% , Area 2 - 0% , Area 3 - 0%
Humidity (%): 17.00 %
Temperature (°C): 25.30 °C
East Facing light detected, good for watering
Rain detected! Turning off all pumps and suspending sensors to avoid damage.
Pump 1 OFF
Pump 2 OFF
Pump 3 OFF
Rain cleared, resuming normal operation.
No rain detected
Soil Moisture (%):
Area 1 - 0% , Area 2 - 0% , Area 3 - 0%
Humidity (%): 17.00 %
Temperature (°C): 25.30 °C
East Facing light detected, good for watering
Pump 1 ON
Pump 2 ON
Pump 3 ON
```

2. Code before multitasking on the Arduino (2.1 – 2.6)

```
// This code is using the DHT Sensor library by Adafruit,
// the Servo library by Michael Margolis & Arduino,
// and the LiquidCrystal_I2C library by John Rickman
// [https://github.com/adafruit/DHT-sensor-library]
// [https://docs.arduino.cc/libraries/servo/]
// [https://github.com/johnrickman/LiquidCrystal_I2C]
#include "DHT.h"
#include <Servo.h>
#include <LiquidCrystal_I2C.h>
#include <Wire.h>

// ---------------- PIN DEFINITIONS ----------------
// Analog inputs
#define AREA_ONE A0
#define AREA_TWO A1
#define AREA_THREE A2
#define LIGHT_PIN A3

// Digital inputs
#define HUMID_TEMP_PIN 2
#define DHT_TYPE DHT11
```

```
const int RELAY_PINS[3] = { 6, 9, 10 };
const int RAIN_PIN = 5;
const int SERVO_PIN = 3;
const int WATER_SWITCH_PIN = 4;
const int MAX_WATER_SECONDS = 30;

// ---------------- OBJECTS ----------------
DHT dht(HUMID_TEMP_PIN, DHT_TYPE);
Servo pumpIndicator;
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ---------------- GLOBAL VARIABLES ----------------
int areas[3];         // Raw soil sensor values
int soilMoisture[3];  // Soil moisture percentage values
int lightPercent;     // Light % value
bool isRain = false;

// Relay (pump) control
const bool RELAY_ACTIVE_LOW = true;
bool pumpStates[3] = { false, false, false };
const int PUMP_ON_THRESHOLD = 40;   // soil too dry below 40%
const int PUMP_OFF_THRESHOLD = 60;  // stop watering once soil reaches 60%

// Calibration values for soil moisture sensors
const int DRY_SOIL[3] = { 552, 537, 535 };  // average dry values per sensor
const int WET_SOIL[3] = { 445, 405, 373 };  // estimated fully wet values per sensor

// Calibration values for the LDR sensor
const int DARK = 900;
const int BRIGHT = 30;

// ---------------- FUNCTIONS ----------------
// Inline functions for controlling pumps
inline void pumpOn(int pumpIndex) {
  digitalWrite(RELAY_PINS[pumpIndex], RELAY_ACTIVE_LOW ? LOW : HIGH);
  pumpStates[pumpIndex] = true;
}
inline void pumpOff(int pumpIndex) {
  digitalWrite(RELAY_PINS[pumpIndex], RELAY_ACTIVE_LOW ? HIGH : LOW);
  pumpStates[pumpIndex] = false;
}

// Convert temperature (°C) and RH (%) to VPD (kPa)
float calcVPD(float tempC, float RH) {
  float svp = 0.6108 * exp((17.27 * tempC) / (tempC + 237.3));  // saturation vapor
pressure
  float avp = svp * (RH / 100.0);                          // actual vapor pressure
  return svp - avp;                                   // VPD
}
```

```
// Terminate processes when rain detected
void emergencyStopRain() {
  Serial.println("Rain detected! Turning off all pumps and suspending sensors.");
  for (int i = 0; i < 3; i++) pumpOff(i);

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("No active pumps");
  lcd.setCursor(0, 1);
  lcd.print("Rain detected!");

  isRain = true;
  while (digitalRead(RAIN_PIN) == LOW) {
    Serial.println("Rain sensor wet...");
    delay(1000);
  }
  isRain = false;  // only reset when rain stops
  Serial.println("Rain cleared, resuming normal operation.");
}

void emergencyStopTemp() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("No active pumps");
  lcd.setCursor(0, 1);
  lcd.print("Sensor Error!");

  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();

  while (isnan(humidity) || isnan(temperature)) {
    Serial.println("Waiting for DHT11...");
    delay(2000);
    humidity = dht.readHumidity();
    temperature = dht.readTemperature();
  }

  Serial.println("DHT11 recovered, resuming operation.");
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("System Ready");
  lcd.setCursor(0, 1);
  lcd.print("No active pumps");
}

// Check for raindrops
void checkRain() {
  bool isWet = (digitalRead(RAIN_PIN) == LOW);
  if (!isRain && isWet) {
    isRain = true;
```

```
    emergencyStopRain();
  }
}

// ---------------- SETUP ----------------
void setup() {
  Serial.begin(9600);
  dht.begin();

  // Setup pump relay pins and ensure OFF
  for (int i = 0; i < 3; i++) {
    pinMode(RELAY_PINS[i], OUTPUT);
    pumpOff(i);
  }

  // Setup sensor pins
  pinMode(AREA_ONE, INPUT);
  pinMode(AREA_TWO, INPUT);
  pinMode(AREA_THREE, INPUT);
  pinMode(LIGHT_PIN, INPUT);
  pinMode(RAIN_PIN, INPUT);
  pumpIndicator.attach(SERVO_PIN);
  pumpIndicator.write(0);

  // Setup 16x2 LCD
  lcd.init();
  lcd.backlight();
  delay(1000);
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("System Ready");
  lcd.setCursor(0, 1);
  lcd.print("No active pumps");
  delay(1000);
}

// ---------------- MAIN LOOP ----------------
void loop() {
  // -------- Rain Detection --------
  checkRain();

  // -------- Soil Moisture --------
  areas[0] = analogRead(AREA_ONE);
  areas[1] = analogRead(AREA_TWO);
  areas[2] = analogRead(AREA_THREE);

  for (int i = 0; i < 3; i++) {
    soilMoisture[i] = map(areas[i], DRY_SOIL[i], WET_SOIL[i], 0, 100);
    soilMoisture[i] = constrain(soilMoisture[i], 0, 100);
  }
```

```
  Serial.println("Soil Moisture (%):");
  Serial.print("Area 1 - ");
  Serial.print(soilMoisture[0]);
  Serial.print("% , Area 2 - ");
  Serial.print(soilMoisture[1]);
  Serial.print("% , Area 3 - ");
  Serial.print(soilMoisture[2]);
  Serial.println("%");

  // -------- Humidity & Temperature --------
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();

  if (isnan(humidity) || isnan(temperature)) {
    emergencyStopTemp();
    return;  // skip rest of loop if DHT failed
  }

  Serial.print("Humidity (%): ");
  Serial.print(humidity);
  Serial.println(" %");
  Serial.print("Temperature (°C): ");
  Serial.print(temperature);
  Serial.println(" °C");

  // -------- Light Sensor --------
  lightPercent = map(analogRead(LIGHT_PIN), DARK, BRIGHT, 0, 100);
  lightPercent = constrain(lightPercent, 0, 100);

  if (lightPercent >= 70) {
    Serial.println("East Facing light detected, good for watering");
  } else if (lightPercent >= 40) {
    Serial.println("North Facing light, not optimal for watering");
  } else {
    Serial.println("Evening light detected, no watering advised");
  }

  // -------- Watering Conditions --------
  bool isLightOK = (lightPercent >= 70);

  // -------- VPD Adjustment --------
  float vpd = calcVPD(temperature, humidity);  // kPa
  // adjust soil moisture threshold based on VPD
  int effectivePumpOnThreshold = PUMP_ON_THRESHOLD - (int)((vpd - 1.0) * 10);
  effectivePumpOnThreshold = constrain(effectivePumpOnThreshold, 30,
PUMP_ON_THRESHOLD);
  Serial.println(effectivePumpOnThreshold);

  if (!isLightOK) {
```

```
  Serial.println("Light too low, no watering (fungal risk)");
}

// --- Per-Area Pump Control with 30s max watering ---
bool watering[3] = { false, false, false };
int activePumps = 0;
for (int i = 0; i < 3; i++) {
  if (isLightOK && soilMoisture[i] <= effectivePumpOnThreshold) {
    pumpOn(i);
    watering[i] = true;
    activePumps++;
  } else {
    pumpOff(i);
    watering[i] = false;
  }
}

// Only water if at least one pump is needed
if (activePumps > 0) {
  int secondsRemaining = MAX_WATER_SECONDS;
  int sensorPins[3] = { AREA_ONE, AREA_TWO, AREA_THREE };

  while (secondsRemaining > 0 && activePumps > 0) {
    activePumps = 0;

    // -------- Rain check inside watering loop --------
    if (digitalRead(RAIN_PIN) == LOW) {  // rain detected
      Serial.println("Rain detected, stopping watering!");
      for (int i = 0; i < 3; i++) {
        pumpOff(i);
        watering[i] = false;
      }
      activePumps = 0;

      // Immediately update LCD to show rain stop
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("No active pumps");
      lcd.setCursor(0, 1);
      lcd.print("Rain detected!");
      delay(2000);
      break;  // exit watering loop fully
    }

    // Update soil moisture and stop pumps if threshold reached
    for (int i = 0; i < 3; i++) {
      if (watering[i]) {
        areas[i] = analogRead(sensorPins[i]);
        soilMoisture[i] = map(areas[i], DRY_SOIL[i], WET_SOIL[i], 0, 100);
        soilMoisture[i] = constrain(soilMoisture[i], 0, 100);
```

```
    if (soilMoisture[i] >= PUMP_OFF_THRESHOLD) {
      pumpOff(i);
      watering[i] = false;
    } else {
      activePumps++;
    }
  }
}

// Update servo indicator
switch (activePumps) {
  case 0: pumpIndicator.write(0); break;
  case 1: pumpIndicator.write(60); break;
  case 2: pumpIndicator.write(120); break;
  case 3: pumpIndicator.write(180); break;
}

// --- Always refresh LCD top row ---
lcd.setCursor(0, 0);
String topText = "";
int count = 0;
for (int i = 0; i < 3; i++)
  if (watering[i]) count++;
if (count == 0) topText = "No active pumps";
else if (count == 3) topText = "All areas";
else {
  for (int i = 0; i < 3; i++) {
    if (watering[i]) {
      if (topText.length() > 0) topText += ",";
      topText += String(i + 1);
    }
  }
  topText = "Area " + topText;
}
topText += "        ";  // pad to clear old characters
lcd.print(topText);

// -------- Update LCD bottom row --------
lcd.setCursor(0, 1);
lcd.print("Time left: " + String(secondsRemaining) + "s   ");

delay(1000);
secondsRemaining--;

// -------- Re-evaluate light sensor --------
lightPercent = map(analogRead(LIGHT_PIN), DARK, BRIGHT, 0, 100);
lightPercent = constrain(lightPercent, 0, 100);
isLightOK = (lightPercent >= 70);

if (!isLightOK) {
```

```
        Serial.println("Light too low, stopping watering!");
        for (int i = 0; i < 3; i++) pumpOff(i);
        activePumps = 0;

        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("No active pumps");
        lcd.setCursor(0, 1);
        lcd.print("Light not good");
        delay(2000);
        break;
      }
    }
  }

  // Ensure all pumps are off at the end
  for (int i = 0; i < 3; i++) pumpOff(i);
  pumpIndicator.write(0);

  // Clear bottom row
  lcd.setCursor(0, 1);
  lcd.print("                ");

  delay(2000);
}
```

APPENDIX C: FINAL DEVELOPED CODE

1. Final Arduino Code

```
// ---------------- LIBRARY INCLUDES ----------------
// This code uses the DHT Sensor library by Adafruit,
// the Servo library by Michael Margolis & Arduino,
// and the LiquidCrystal_I2C library by John Rickman
#include "DHT.h"
#include <Servo.h>
#include <LiquidCrystal_I2C.h>
#include <Wire.h>

// ---------------- PIN DEFINITIONS ----------------
// Analog inputs
#define AREA_ONE A0
#define AREA_TWO A1
#define AREA_THREE A2
#define LIGHT_PIN A3

// Digital inputs
#define HUMID_TEMP_PIN 2
#define DHT_TYPE DHT11
#define RAIN_PIN 5

// Digital outputs
const int RELAY_PINS[3] = { 6, 9, 10 };  // Pump relay pins
const int SERVO_PIN = 3;                 // Servo indicator pin

// ---------------- WATERING PARAMETERS ----------------
const int MAX_WATER_SECONDS = 30;          // Max watering time per area
const int PUMP_ON_THRESHOLD = 40;          // Start watering if soil below this (%)
const int PUMP_OFF_THRESHOLD = 60;         // Stop watering if soil above this
(%)
const int DRY_SOIL[3] = { 552, 537, 555 };  // ADC value for dry soil
const int WET_SOIL[3] = { 520, 430, 395 };  // ADC value for wet soil
const int DARK = 30;                       // ADC value for very low light
const int BRIGHT = 900;                    // ADC value for very bright light
const bool RELAY_ACTIVE_LOW = true;        // true if relay is active-low

// ---------------- OBJECTS ----------------
DHT dht(HUMID_TEMP_PIN, DHT_TYPE);   // DHT temperature & humidity sensor
Servo pumpIndicator;                 // Servo for pump activity indication
LiquidCrystal_I2C lcd(0x27, 16, 2);  // 16x2 LCD display

// ---------------- GLOBAL VARIABLES ----------------
// Sensor readings
int areas[3];         // Raw soil sensor readings
int soilMoisture[3];  // Soil moisture percentages
int lightPercent;     // Light percentage
```

```
// Environment status flags
bool isLightBad = false;
bool isRain = false;
bool dhtError = false;

// Pump states
bool pumpStates[3] = { false, false, false };  // Relay ON/OFF
bool watering[3] = { false, false, false };    // Active watering flag
int secondsRemaining = MAX_WATER_SECONDS;      // Countdown for watering

// Pump cycle counters
int pumpCounters[3] = { 0, 0, 0 };  // Number of completed watering cycles

// Maintenance flag (set by Pi)
bool maintenanceActive = false;

// Timing variables
unsigned long lastPumpUpdate = 0;
unsigned long lastDHTUpdate = 0;
unsigned long lastLCDUpdate = 0;
unsigned long lastSensorCheck = 0;
unsigned long wateringStartTime[3] = { 0, 0, 0 };  // Track start time per area
const unsigned long pumpInterval = 1000;           // Pump update interval
const unsigned long dhtInterval = 2000;            // DHT read interval
const unsigned long lcdInterval = 500;             // LCD refresh interval
const unsigned long sensorCheckInterval = 200;     // Light/rain check interval
unsigned long lastPrint = 0;                       // Last serial print time

// DHT sensor readings
float humidity = 0;
float temperature = 0;

// ---------------- PREVIOUS VALUES FOR SERIAL THRESHOLDS ----------------
int prevSoil[3] = { -1, -1, -1 };
float prevTemp = -1000;
float prevHumidity = -1;
bool prevRain = false;
int prevLight = -1;
bool prevPump[3] = { false, false, false };

// Thresholds for significant changes (for serial printing)
const int SOIL_THRESHOLD = 2;
const float TEMP_THRESHOLD = 0.5;
const float HUM_THRESHOLD = 1.0;
const int LIGHT_THRESHOLD = 5;
bool shouldPrint = false;
bool serialAborted = false;  // Block serial printing during alerts

// ---------------- HELPER FUNCTIONS ----------------
```

```
// Turn pump ON
inline void pumpOn(int i) {
  digitalWrite(RELAY_PINS[i], RELAY_ACTIVE_LOW ? LOW : HIGH);
  pumpStates[i] = true;
}

// Turn pump OFF
inline void pumpOff(int i) {
  digitalWrite(RELAY_PINS[i], RELAY_ACTIVE_LOW ? HIGH : LOW);
  pumpStates[i] = false;
}

// Check DHT sensor readings
void checkDHT() {
  float h = dht.readHumidity();
  float t = dht.readTemperature();
  if (isnan(h) || isnan(t)) {
    dhtError = true;
  } else {
    humidity = h;
    temperature = t;
    dhtError = false;
  }
}

// Check rain sensor
void checkRain() {
  isRain = (digitalRead(RAIN_PIN) == LOW);
}

// Check light sensor
void checkLight() {
  lightPercent = map(analogRead(LIGHT_PIN), DARK, BRIGHT, 0, 100);
  lightPercent = constrain(lightPercent, 0, 100);
  isLightBad = (lightPercent < 70);
}

// Read and update soil moisture
void updateSoilMoisture() {
  for (int i = 0; i < 3; i++) {
    areas[i] = analogRead(i == 0 ? AREA_ONE : (i == 1 ? AREA_TWO :
AREA_THREE));
    soilMoisture[i] = map(areas[i], DRY_SOIL[i], WET_SOIL[i], 0, 100);
    soilMoisture[i] = constrain(soilMoisture[i], 0, 100);
  }
}

// Update pumps based on conditions and maintenance
void updatePumps() {
  int activePumps = 0;
```

```
  for (int i = 0; i < 3; i++) {

    // If maintenance is active, turn all pumps OFF
    if (maintenanceActive) {
      pumpOff(i);
      watering[i] = false;
      continue;
    }

    bool shouldStart = (!dhtError && !isRain && !isLightBad && soilMoisture[i] <=
PUMP_ON_THRESHOLD);
    bool shouldStop = (soilMoisture[i] >= PUMP_OFF_THRESHOLD || millis() -
wateringStartTime[i] >= MAX_WATER_SECONDS * 1000UL);

    // Start pump if needed
    if (shouldStart && !pumpStates[i]) {
      pumpOn(i);
      watering[i] = true;
      wateringStartTime[i] = millis();
      pumpCounters[i]++;
    }

    // Stop pump if needed
    if (pumpStates[i] && (shouldStop || dhtError || isRain || isLightBad)) {
      pumpOff(i);
      watering[i] = false;
    }

    if (pumpStates[i]) activePumps++;
  }

  // Update servo to indicate active pumps
  switch (activePumps) {
    case 0: pumpIndicator.write(0); break;
    case 1: pumpIndicator.write(60); break;
    case 2: pumpIndicator.write(120); break;
    case 3: pumpIndicator.write(180); break;
  }

  // Countdown timer
  static unsigned long lastSecond = 0;
  if (activePumps > 0 && millis() - lastSecond >= 1000) {
    secondsRemaining--;
    lastSecond = millis();
    if (secondsRemaining <= 0) {
      for (int i = 0; i < 3; i++) pumpOff(i);
      for (int i = 0; i < 3; i++) watering[i] = false;
      secondsRemaining = MAX_WATER_SECONDS;
    }
```

```
  } else if (activePumps == 0) {
    secondsRemaining = MAX_WATER_SECONDS;
  }
}

// Update LCD display
void updateLCD() {
  lcd.setCursor(0, 0);
  String topText = "";
  String bottomText = "";

  if (maintenanceActive) {
    topText = "MAINTENANCE    ";
    bottomText = "Pumps stopped ";
  } else if (dhtError) {
    topText = "DHT error      ";
    bottomText = "Pumps terminated";
  } else if (isRain) {
    topText = "Rain detected  ";
    bottomText = "Remove System! ";
  } else if (isLightBad) {
    topText = "Low/No Light  ";
    bottomText = "Pumps terminated";
  } else {
    int count = 0;
    for (int i = 0; i < 3; i++)
      if (watering[i]) count++;

    if (count == 0) {
      topText = "No active pumps";
      bottomText = "                ";
    } else if (count == 3) {
      topText = "All areas      ";
      bottomText = "Time left: " + String(secondsRemaining) + "s";
    } else {
      topText = "Area ";
      bool first = true;
      for (int i = 0; i < 3; i++) {
        if (watering[i]) {
          if (!first) topText += ",";
          topText += String(i + 1);
          first = false;
        }
      }
      bottomText = "Time left: " + String(secondsRemaining) + "s";
    }
  }

  while (topText.length() < 16) topText += " ";
  while (bottomText.length() < 16) bottomText += " ";
```

```
  lcd.setCursor(0, 0);
  lcd.print(topText);
  lcd.setCursor(0, 1);
  lcd.print(bottomText);
}

// Print sensor values on Serial only on significant change
void printIfChanged() {
  bool abortCondition = dhtError || isRain || isLightBad;

  if (abortCondition) {
    if (!serialAborted) {
      Serial.print("SYSTEM ALERT: ");
      if (dhtError) Serial.println("DHT error - Pumps terminated");
      else if (isRain) Serial.println("Rain detected - Pumps terminated");
      else if (isLightBad) Serial.println("Low/No light - Pumps terminated");

      Serial.print("Pump Cycles - ");
      for (int i = 0; i < 3; i++) {
        Serial.print(pumpCounters[i]);
        if (i < 2) Serial.print(" , ");
      }
      Serial.println();
      serialAborted = true;
    }
    return;
  } else serialAborted = false;

  shouldPrint = false;

  // Soil, temp, humidity, rain, light, pump changes
  for (int i = 0; i < 3; i++)
    if (abs(soilMoisture[i] - prevSoil[i]) >= SOIL_THRESHOLD) {
      shouldPrint = true;
      prevSoil[i] = soilMoisture[i];
    }
  if (abs(temperature - prevTemp) >= TEMP_THRESHOLD) {
    shouldPrint = true;
    prevTemp = temperature;
  }
  if (abs(humidity - prevHumidity) >= HUM_THRESHOLD) {
    shouldPrint = true;
    prevHumidity = humidity;
  }
  if (isRain != prevRain) {
    shouldPrint = true;
    prevRain = isRain;
  }
  if (abs(lightPercent - prevLight) >= LIGHT_THRESHOLD) {
    shouldPrint = true;
```

```
    prevLight = lightPercent;
  }
  for (int i = 0; i < 3; i++)
    if (pumpStates[i] != prevPump[i]) {
      shouldPrint = true;
      prevPump[i] = pumpStates[i];
    }

  // Print if significant change detected
  if (shouldPrint) {
    Serial.print("Soil:");
    for (int i = 0; i < 3; i++) {
      Serial.print(soilMoisture[i]);
      if (i < 2) Serial.print(",");
    }
    Serial.print(" | DHT: ");
    Serial.print(temperature);
    Serial.print("C,");
    Serial.print(humidity);
    Serial.print("%");
    Serial.print(" | Rain: ");
    Serial.print(isRain ? "YES" : "NO");
    Serial.print(" | Light: ");
    Serial.print(lightPercent);
    Serial.print("%");
    Serial.print(" | Pumps: ");
    for (int i = 0; i < 3; i++) {
      Serial.print(pumpStates[i] ? "ON" : "OFF");
      if (i < 2) Serial.print(",");
    }
    Serial.print(" | Pump Cycles - ");
    for (int i = 0; i < 3; i++) {
      Serial.print(pumpCounters[i]);
      if (i < 2) Serial.print(" , ");
    }
    Serial.println();
  }
}

// ---------------- SETUP ----------------
void setup() {
  Serial.begin(9600);
  dht.begin();

  // Initialise relay pins
  for (int i = 0; i < 3; i++) {
    pinMode(RELAY_PINS[i], OUTPUT);
    pumpOff(i);
  }
```

```
  // Initialise analog/digital sensors
  pinMode(AREA_ONE, INPUT);
  pinMode(AREA_TWO, INPUT);
  pinMode(AREA_THREE, INPUT);
  pinMode(LIGHT_PIN, INPUT);
  pinMode(RAIN_PIN, INPUT);

  // Initialise servo
  pumpIndicator.attach(SERVO_PIN);
  pumpIndicator.write(0);

  // Initialise LCD
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("System Ready");
  lcd.setCursor(0, 1);
  lcd.print("No active pumps");
}

// ---------------- MAIN LOOP ----------------
void loop() {
  unsigned long now = millis();

  // ---------------- SERIAL HANDLING ----------------
  if (Serial.available() > 0) {
    String line = Serial.readStringUntil('\n');

    // Reset pump counters
    if (line.startsWith("RESET:")) {
      line.remove(0, 6);
      int newCounters[3] = { 0 };
      int index = 0;
      char *token = strtok((char *)line.c_str(), ",");
      while (token && index < 3) {
        newCounters[index++] = atoi(token);
        token = strtok(NULL, ",");
      }
      for (int i = 0; i < 3; i++) pumpCounters[i] = newCounters[i];
      maintenanceActive = false;
    }

    // Maintenance commands
    if (line == "MAINT:ON") {
      maintenanceActive = true;
      for (int i = 0; i < 3; i++) pumpOff(i);
    }
    if (line == "MAINT:OFF") { maintenanceActive = false; }
  }
```

```
  // ---------------- SENSOR UPDATES ----------------
  if (now - lastPumpUpdate >= pumpInterval) {
    lastPumpUpdate = now;
    updateSoilMoisture();
    updatePumps();
  }
  if (now - lastDHTUpdate >= dhtInterval) {
    lastDHTUpdate = now;
    checkDHT();
  }
  if (now - lastSensorCheck >= sensorCheckInterval) {
    lastSensorCheck = now;
    checkRain();
    checkLight();
  }

  // ---------------- LCD UPDATE ----------------
  if (now - lastLCDUpdate >= lcdInterval) {
    lastLCDUpdate = now;
    updateLCD();
  }

  // ---------------- SERIAL PRINT ----------------
  if (now - lastPrint >= 500) {
    lastPrint = now;
    printIfChanged();
  }
}
```

2. Final Raspberry Pi C++ Code

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <atomic>
#include <mutex>
#include <thread>
#include <chrono>
#include <csignal>
#include <cmath>
#include <deque>
#include <termios.h>
#include <fcntl.h>
#include <unistd.h>
#include <pigpio.h>

// ======================== PIR / ULTRASONIC / BUZZER
========================
#define PIR_PIN     17
#define BUZZER_PIN  18
#define US_TRIG     23
#define US_ECHO     24

const uint64_t PIR_HOLD_TIME_US   = 4000000; // 4 s
const double   US_DISTANCE_LIMIT   = 150.0;   // cm
const int      US_SMOOTH_SAMPLES   = 5;       // moving average
const double   US_PRINT_THRESHOLD  = 5.0;     // cm diff to print

// ======================== PUMP SUPERVISOR / ARDUINO
========================
#define LED_PIN     27
#define BUTTON_PIN  22
static const std::string SERIAL_PORT = "/dev/ttyACM0";
static const int BAUD_RATE = B9600;
static const std::string COUNTER_FILE = "pump_states.txt";
static const int MAINTENANCE_THRESHOLD = 30;

// ======================== GLOBAL STATE (shared)
========================
struct Counters { int a=0, b=0, c=0; };

static std::atomic<bool> running(true);
static std::atomic<bool> maintenanceActive(false);
static std::mutex stateMtx;         // protects 'latest' and file writes
static Counters latest;

static std::atomic<uint64_t> buzzerEndTime(0); // end time for buzzer ON window
static std::atomic<double> averageDistance(999.0);
```

```
static std::mutex printMutex;

static int serialFd = -1;

// ======================== FORWARD DECLARATIONS
========================
// Signal
void signalHandler(int signum);

// PIR/Ultrasonic/Buzzer
double readUltrasonic();
void   ultrasonicThread();
void   motionThread();
void   buzzerThread();

// Serial/Arduino utils
bool writeAll(int fd, const std::string &s);
bool sendToArduino(const std::string& line);
void setMaintenanceLED(bool on);
int  openSerial(const std::string& port, int baudFlag);

// File + parse
void saveCountersToFile(const Counters& c, bool maint);
bool parsePumpCycles(const std::string& line, Counters& out);

// Threads / callbacks
void serialReaderThread();
void watchdogThread();
void buttonCallback(int gpio, int level, uint32_t tick);

// ======================== IMPLEMENTATION ========================

// ---- Signal ----
void signalHandler(int) { running = false; }

// ---- Ultrasonic distance (cm) ----
double readUltrasonic() {
    gpioWrite(US_TRIG, 0);
    gpioDelay(2);              // 2 µs low
    gpioWrite(US_TRIG, 1);
    gpioDelay(10);             // 10 µs pulse
    gpioWrite(US_TRIG, 0);

    uint32_t startTick = gpioTick();
    uint32_t timeout   = startTick + 30000; // 30 ms timeout

    while (gpioRead(US_ECHO) == 0 && gpioTick() < timeout) {}
    if (gpioTick() >= timeout) return -1;

    uint32_t echoStart = gpioTick();
```

```
    while (gpioRead(US_ECHO) == 1 && gpioTick() < timeout) {}
    if (gpioTick() >= timeout) return -1;

    uint32_t echoEnd = gpioTick();
    double pulseDuration = static_cast<double>(echoEnd - echoStart); // µs
    return pulseDuration / 58.0; // HC-SR04 approximate conversion to cm
}

// ---- Ultrasonic thread ----
void ultrasonicThread() {
    std::deque<double> distanceBuffer;
    double lastPrintedDistance = -1.0;

    while (running) {
        double d = readUltrasonic();
        if (d > 0 && d <= 300.0) { // valid range
            distanceBuffer.push_back(d);
            if (distanceBuffer.size() > US_SMOOTH_SAMPLES)
distanceBuffer.pop_front();

            double sum = 0;
            for (auto v : distanceBuffer) sum += v;
            double avg = sum / distanceBuffer.size();
            averageDistance.store(avg);

            if (std::fabs(avg - lastPrintedDistance) >= US_PRINT_THRESHOLD) {
                std::lock_guard<std::mutex> lock(printMutex);
                std::cout << "Ultrasonic avg distance: " << avg << " cm" << std::endl;
                lastPrintedDistance = avg;
            }
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(50));
    }
}

// ---- PIR motion thread ----
void motionThread() {
    while (running) {
        if (gpioRead(PIR_PIN) == 1) {
            uint64_t now = gpioTick();
            double dist = averageDistance.load();
            std::lock_guard<std::mutex> lock(printMutex);
            if (dist <= US_DISTANCE_LIMIT) {
                buzzerEndTime.store(now + PIR_HOLD_TIME_US);
                std::cout << "Motion + Object detected at " << dist << " cm" << std::endl;
            } else {
                std::cout << "Motion detected but object too far (" << dist << " cm)" <<
std::endl;
            }
```

```
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(50));
    }
}

// ---- Buzzer thread ----
void buzzerThread() {
    bool buzzerWasOn = false;

    while (running) {
        bool buzzerOn = (gpioTick() < buzzerEndTime.load());
        if (buzzerOn != buzzerWasOn) {
            if (buzzerOn) gpioHardwarePWM(BUZZER_PIN, 2000, 500000); // 2kHz,
50%
            else          gpioHardwarePWM(BUZZER_PIN, 0, 0);         // stop
            buzzerWasOn = buzzerOn;

            std::lock_guard<std::mutex> lock(printMutex);
            std::cout << (buzzerOn ? "Buzzer ON" : "Buzzer OFF") << std::endl;
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(50));
    }
    gpioHardwarePWM(BUZZER_PIN, 0, 0);
}

// ---- Serial helpers ----
bool writeAll(int fd, const std::string &s) {
    const char* p = s.c_str();
    size_t left = s.size();
    while (left) {
        ssize_t n = ::write(fd, p, left);
        if (n < 0) {
            if (errno == EINTR) continue;
            perror("write");
            return false;
        }
        left -= (size_t)n;
        p += n;
    }
    return true;
}

bool sendToArduino(const std::string& line) {
    if (serialFd < 0) return false;
    return writeAll(serialFd, line);
}

void setMaintenanceLED(bool on) { gpioWrite(LED_PIN, on ? 1 : 0); }

int openSerial(const std::string& port, int baudFlag) {
```

```
    int fd = ::open(port.c_str(), O_RDWR | O_NOCTTY | O_NONBLOCK);
    if (fd < 0) { perror("open serial"); return -1; }

    termios tio{};
    if (tcgetattr(fd, &tio) != 0) { perror("tcgetattr"); ::close(fd); return -1; }
    cfmakeraw(&tio);
    cfsetispeed(&tio, baudFlag);
    cfsetospeed(&tio, baudFlag);
    tio.c_cflag |= (CLOCAL | CREAD);
    tio.c_cflag &= ~PARENB;
    tio.c_cflag &= ~CSTOPB;
    tio.c_cflag &= ~CSIZE;
    tio.c_cflag |= CS8;
    tio.c_cc[VMIN]  = 0;  // non-blocking
    tio.c_cc[VTIME] = 0;

    if (tcsetattr(fd, TCSANOW, &tio) != 0) { perror("tcsetattr"); ::close(fd); return -1; }

    // switch to blocking for simpler line assembly
    int flags = fcntl(fd, F_GETFL, 0);
    fcntl(fd, F_SETFL, flags & ~O_NONBLOCK);
    return fd;
}

// ---- File + Parse ----
void saveCountersToFile(const Counters& c, bool maint) {
    std::ofstream ofs(COUNTER_FILE, std::ios::out | std::ios::trunc);
    if (!ofs) return;
    auto ts = std::chrono::system_clock::to_time_t(std::chrono::system_clock::now());
    ofs << "ts=" << ts
        << " a=" << c.a
        << " b=" << c.b
        << " c=" << c.c
        << " maintenance=" << (maint ? 1 : 0)
        << "\n";
}

bool parsePumpCycles(const std::string& line, Counters& out) {
    auto pos = line.find("Pump Cycles -");
    if (pos == std::string::npos) return false;
    std::string tail = line.substr(pos + std::string("Pump Cycles -").size());

    // normalize commas to spaces
    for (char& ch : tail) if (ch == ',') ch = ' ';

    std::istringstream iss(tail);
    int values[3], found = 0; std::string tok;
    while (iss >> tok) {
        char* endptr = nullptr;
        long v = std::strtol(tok.c_str(), &endptr, 10);
```

```
        if (*endptr == '\0') { if (found < 3) values[found++] = (int)v; if (found == 3) break; }
    }
    if (found == 3) { out.a = values[0]; out.b = values[1]; out.c = values[2]; return true; }
    return false;
}

// ---- Serial reader thread ----
void serialReaderThread() {
    std::string buf; char ch;
    while (running) {
        ssize_t n = ::read(serialFd, &ch, 1);
        if (n == 1) {
            if (ch == '\r') continue;
            if (ch == '\n') {
                if (!buf.empty()) {
                    Counters parsed;
                    if (parsePumpCycles(buf, parsed)) {
                        std::lock_guard<std::mutex> lk(stateMtx);
                        latest = parsed;
                        saveCountersToFile(latest, maintenanceActive.load());
                    }
                }
                buf.clear();
            } else {
                buf.push_back(ch);
                if (buf.size() > 4096) buf.clear();
            }
        } else {
            std::this_thread::sleep_for(std::chrono::milliseconds(10));
        }
    }
}

// ---- Watchdog thread ----
void watchdogThread() {
    while (running) {
        {
            std::lock_guard<std::mutex> lk(stateMtx);
            if (!maintenanceActive) {
                if (latest.a >= MAINTENANCE_THRESHOLD ||
                    latest.b >= MAINTENANCE_THRESHOLD ||
                    latest.c >= MAINTENANCE_THRESHOLD) {
                    sendToArduino("MAINT:ON\n");
                    maintenanceActive = true;
                    setMaintenanceLED(true);
                    saveCountersToFile(latest, true);
                    std::cerr << "[WATCHDOG] MAINT:ON (threshold reached)\n";
                }
            }
        }
```

```
        std::this_thread::sleep_for(std::chrono::milliseconds(200));
    }
}

// ---- Button callback (active on falling edge, pull-up enabled) ----
void buttonCallback(int, int level, uint32_t tick) {
    if (level != 0) return;
    static uint32_t lastTick = 0;
    if (tick - lastTick < 250000) return; // debounce ~250ms
    lastTick = tick;

    {
        std::lock_guard<std::mutex> lk(stateMtx);
        sendToArduino("RESET:0,0,0\n");
        latest = Counters{};
        saveCountersToFile(latest, false);
        sendToArduino("MAINT:OFF\n");
        maintenanceActive = false;
        setMaintenanceLED(false);
    }
    std::cerr << "[BUTTON] Reset sent (RESET:0,0,0) and MAINT:OFF\n";
}

// ======================== MAIN ========================
int main() {
    std::signal(SIGINT, signalHandler);

    if (gpioInitialise() < 0) {
        std::cerr << "Pigpio init failed!" << std::endl;
        return 1;
    }

    // GPIO modes
    gpioSetMode(PIR_PIN, PI_INPUT);
    gpioSetMode(BUZZER_PIN, PI_OUTPUT);
    gpioSetMode(US_TRIG, PI_OUTPUT);
    gpioSetMode(US_ECHO, PI_INPUT);

    gpioSetMode(LED_PIN, PI_OUTPUT);
    gpioSetMode(BUTTON_PIN, PI_INPUT);
    gpioSetPullUpDown(BUTTON_PIN, PI_PUD_UP);
    gpioSetAlertFunc(BUTTON_PIN, buttonCallback);

    setMaintenanceLED(false);

    // Serial open
    serialFd = openSerial(SERIAL_PORT, BAUD_RATE);
    if (serialFd < 0) {
        std::cerr << "Failed to open serial " << SERIAL_PORT << "\n";
        gpioTerminate();
```

```
        return 1;
    }

    // small grace for Arduino after serial open
    std::this_thread::sleep_for(std::chrono::milliseconds(1200));

    std::cout << "PIR + Ultrasonic + Pump Supervisor active.\n"
              << "Waiting for movement & Arduino data..." << std::endl;

    // Threads
    std::thread tUS(ultrasonicThread);
    std::thread tMotion(motionThread);
    std::thread tBuzzer(buzzerThread);

    std::thread tReader(serialReaderThread);
    std::thread tWatch(watchdogThread);

    // Run until SIGINT
    while (running) std::this_thread::sleep_for(std::chrono::milliseconds(200));

    std::cerr << "Shutting down...\n";
    gpioSetAlertFunc(BUTTON_PIN, nullptr);

    if (tUS.joinable()) tUS.join();
    if (tMotion.joinable()) tMotion.join();
    if (tBuzzer.joinable()) tBuzzer.join();
    if (tReader.joinable()) tReader.join();
    if (tWatch.joinable()) tWatch.join();

    if (serialFd >= 0) ::close(serialFd);
    gpioTerminate();

    std::cout << "Program terminated cleanly." << std::endl;
    return 0;
}
```

3. Helper Script To Control Arduino from Pi (Start.sh)

```
#!/bin/bash
echo '1-1' | sudo tee /sys/bus/usb/drivers/usb/bind
sleep 1
sudo ./pi_final
echo '1-1' | sudo tee /sys/bus/usb/drivers/usb/unbind
```

APPENDIX D: SYSTEM DIAGRAMS

Figure 1 – High-Level Block Diagram of Developed System

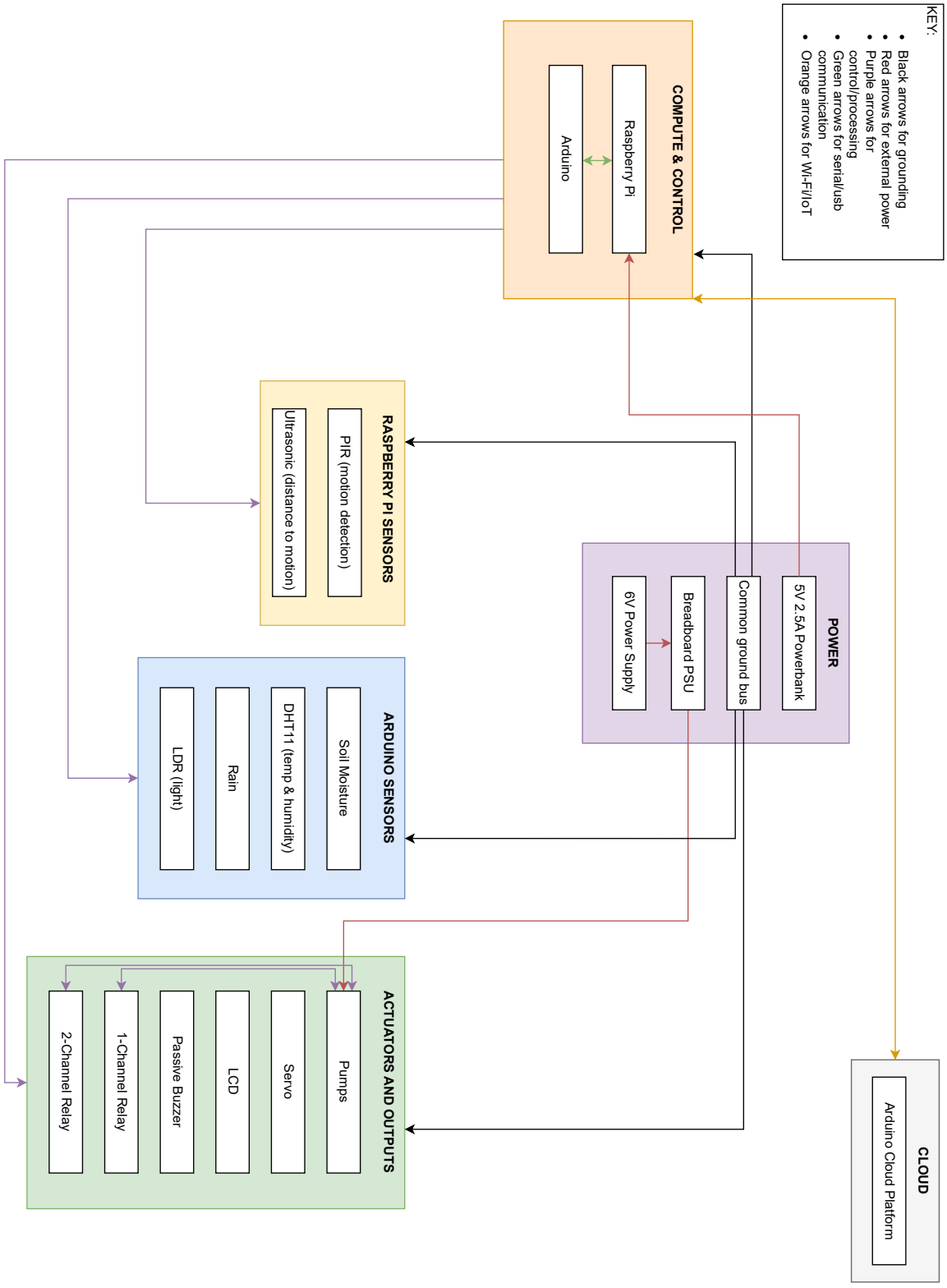


Figure 2 – Circuit Diagram of Developed System

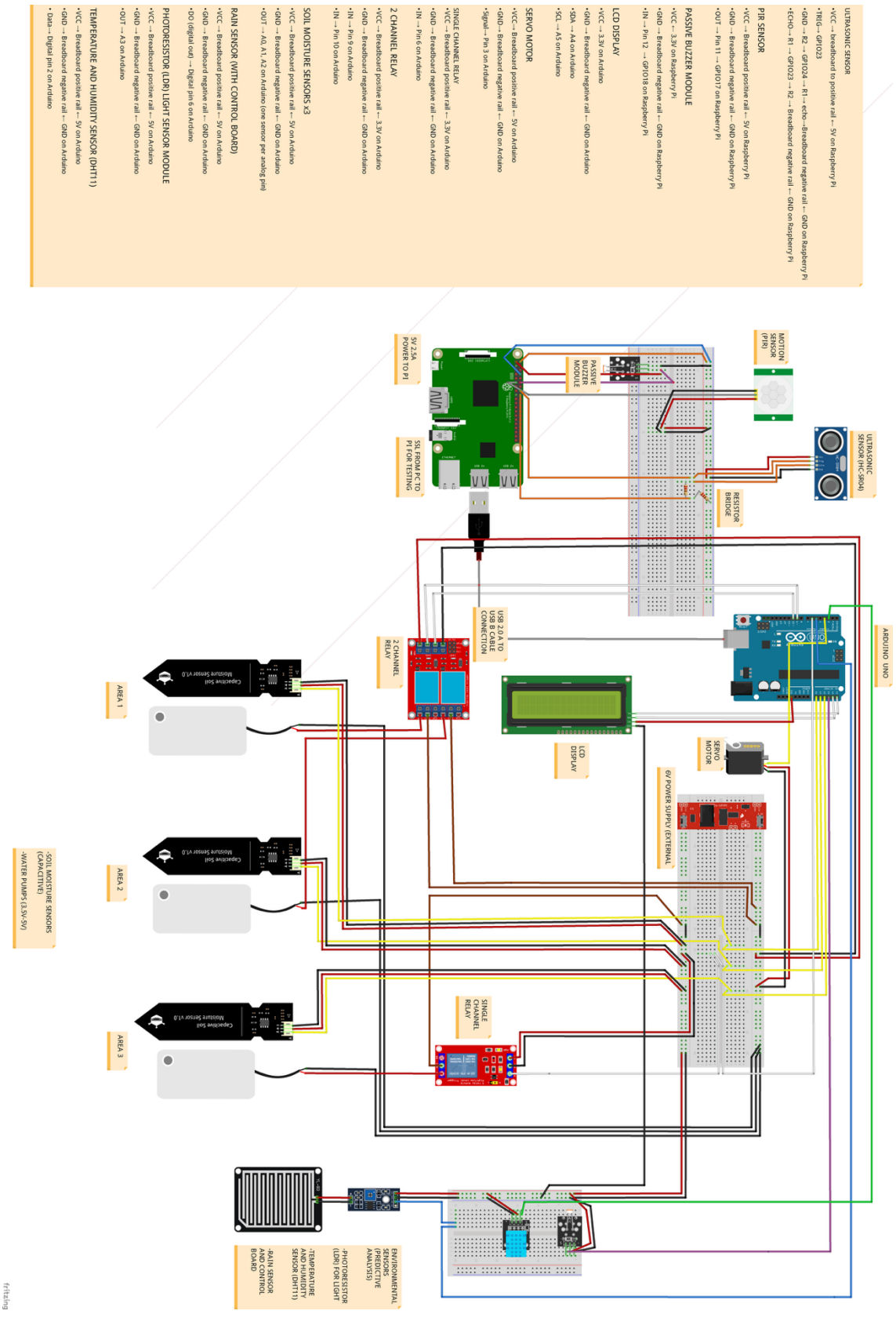
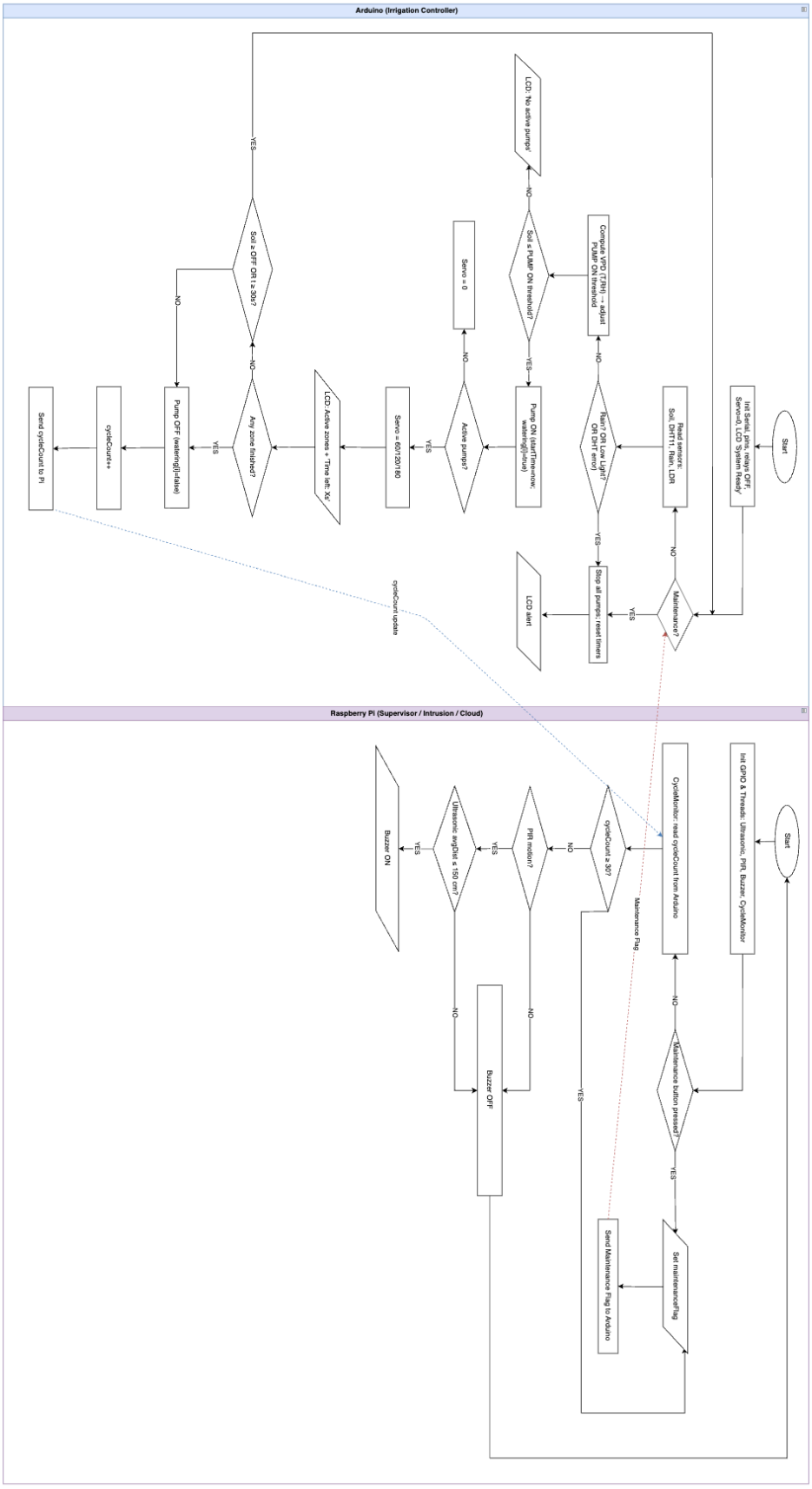


Figure 3 – Flowchart of Developed System



APPENDIX E: KEY ASSUMPTIONS

1. VPD (Vapor Pressure Deficit) For Dynamic Soil Watering

- Used to calculate when pumps should turn on.
- Formula is VPD = SVP x (1 − RH/100) where SVP x (1 − RH%/100) = VPD
- SVP = 610.78 x e^(Temp / (Temp +237.3) x 17.2694)
- Guide found here: https://pulsegrow.com/blogs/learn/vpd?srsltid=AfmBOoq3nw92cRzUqqO0cxdJfTxGo2zdQF8w_Om-sebhOoPUZAztcTvo#vpth

2. Light Conditions

- Midday causes plants to burn and evaporates water. If the LDR is facing eastwards / towards the sun, then midday sun will be on top of the sensor at an angle -> trigger light flag.
- Evening is a good time to water plants, but there is a risk of fungal infection in the soil. Hence, the light will be behind LDR -> trigger light flag.
- Morning is the best time to water plants, LDR is facing direct sun -> trigger pumps when needed.

3. Rain & Snow Detection

- Rain pad triggers a flag at a small drop of water, if a small drizzle has occurred.
- This can also trigger for any water droplet that gets onto the sensor, posing risk for a false positive -> assumptive but adjustable on the module potentiometer.

4. PIR & Ultrasonic Distance Detection

- The PIR is set to the lowest range of 3m in a 120-degree cone.
- The Ultrasonic sensor aims to minimise the distance to an adjustable figure (1.5m in the context of our system), but the cone is shorter than the PIR.